알고리즘 활용

연암공과대학교

Chapter 2

알고리즘을 배우기 위한 준비

연암공과대학교

차례

2.1 알고리즘이란

2.2 최초의 알고리즘

2.3 알고리즘의 표현 방법

2.4 알고리즘의 분류

2.5 알고리즘의 효율성 표현

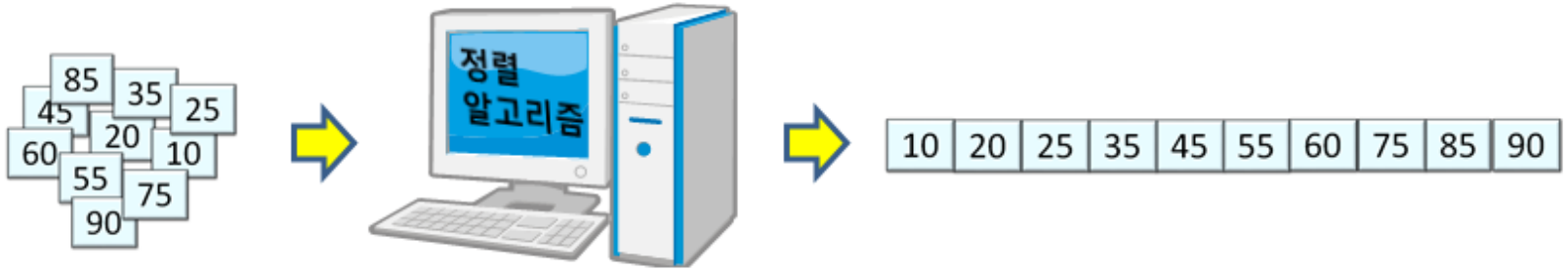
2.6 복잡도의 점근적 표기

2.7 왜 효율적인 알고리즘이 필요한가?

2.1 알고리즘이란

알고리즘

- 문제를 해결하는 단계적 절차 또는 방법
- 여기서 주어지는 문제는 컴퓨터를 이용하여 해결할 수 있어야 함
- 알고리즘에는 입력이 주어지고, 알고리즘은 수행한 결과인 해 (또는 답)를 출력함



알고리즘의 일반적 특성

- 정확성
  - 알고리즘은 주어진 입력에 대해 올바른 해를 주어야(랜덤 알고리즘은 예외)
- 수행성
  - 알고리즘의 각 단계는 컴퓨터에서 수행 가능해야 함
- 유한성
  - 알고리즘은 유한 시간 내에 종료되어야 함
- 효율성
  - 알고리즘은 효율적일수록 그 가치가 높아짐

2.2 최초의 알고리즘

- 유클리드(Euclid)의 최대공약수 알고리즘
  - 기원전 300년경에 만들어진 가장 오래된 알고리즘
  - 최대공약수란 2개의 자연수의 공약수들 중에서 가장 큰 수

2개의 자연수의 최대공약수는 큰 수에서 작은 수를 뺀 수와 작은 수와의 최대공약수와 같다.

최대공약수(24, 14)

최대공약수(24, 14)

= 최대공약수(24-14, 14) = 최대공약수(10, 14)

= 최대공약수(14-10, 10) = 최대공약수(4, 10)

= 최대공약수(10-4, 4) = 최대공약수(6, 4)

= 최대공약수(6-4, 4) = 최대공약수(2, 4)

= 최대공약수(4-2, 2) = 최대공약수(2, 2)

= 최대공약수(2-2, 2) = 최대공약수(0, 2)

= 2

- 유클리드의 최대공약수 알고리즘에서 뺄셈 대신에 나눗셈을 사용하면 빠르게 해를 찾음

알고리즘

Euclid(a, b)

입력: 정수 a, b; **단**, a≥b≥0

출력: **최대공약수**(a, b)

```
1. if b==0 return a
2. return Euclid(b, a mod b)
```

유클리드의 최대공약수 알고리즘

Mod(모듈러) : 양변의 두 수를 서로 나누어서 몫을 제외한 나머지 값을 구하는 나머지 연산 의미
- 문법 : b = mod(a,m)
- 수식 : a mod b

```
Euclid(a, b)
```

입력: **정수** a, b; **단**, a≥b≥0

출력: **최대공약수**(a, b)

```
1.   if b==0 return a
2.   return Euclid(b, a mod b)
```

최대공약수(24, 14)

- **Line 1:** b==14**이므로** if-**조건이 '거짓'**
- **Line 2:** Euclid(14, 24 mod 14) = Euclid(14, 10) **호출**
- **Line 1:** b==10**이므로** if-**조건이 '거짓'**
- **Line 2:** Euclid(10, 14 mod 10) = Euclid(10, 4) **호출**
- **Line 1:** b==4**이므로** if-**조건이 '거짓'**
- **Line 2:** Euclid(4, 10 mod 4) = Euclid(**4, 2) 호출**
- **Line 1:** b==2**이므로** if-**조건이 '거짓'**
- **Line 2:** Euclid(2, 4 mod 2) = Euclid(2, 0) **호출**
- **Line 1:** b==0**이므로** if-**조건이 '참'이 되어** a=2**를 최종 리턴**

유클리드의 최대공약수 알고리즘

➢ Python 프로그램 1

```
def get_gcd_recursive(a, b):
    # 나누어 떨어져서 나머지(b)가 0이 되면 a를 반환합니다.
    if b == 0:
        return a
    # 그렇지 않으면 b와 'a를 b로 나눈 나머지(mod)'를 가지고 다시 함수를 실행합니다.
    else:
        return get_gcd_recursive(b, a % b)

# 테스트
num1 = 60
num2 = 48
print(f"{num1}과 {num2}의 최대공약수: {get_gcd_recursive(num1, num2)}")
```

유클리드의 최대공약수 알고리즘

➢ Python 프로그램 2

```
def get_gcd_with_mod(a, b):
    # b가 0이 아닐 때까지 반복합니다.
    while b != 0:
        # a를 b로 나눈 나머지(mod)를 구합니다.
        remainder = a % b
        # a에는 b값을, b에는 나머지 값을 덮어씌웁니다.
        a = b
        b = remainder
        # 파이썬에서는 위 세 줄을 "a, b = b, a % b" 한 줄로 줄여 쓸 수도 있습니다.
    return a  # b가 0이 되면, 그때의 a가 최대공약수입니다.
# 테스트
num1 = 60
num2 = 48
print(f"{num1}과 {num2}의 최대공약수: {get_gcd_with_mod(num1, num2)}")
```

유클리드의 최대공약수 알고리즘

➢ Python 프로그램 3

```
import math
def get_gcd_builtin(a, b):
        # math.gcd() 함수를 사용해 최대공약수를 반환합니다.
        return math.gcd(a, b)
# 테스트
num1 = 24
num2 = 14
result = get_gcd_builtin(num1, num2)
print(f"{num1}과 {num2}의 최대공약수: {result}")
```

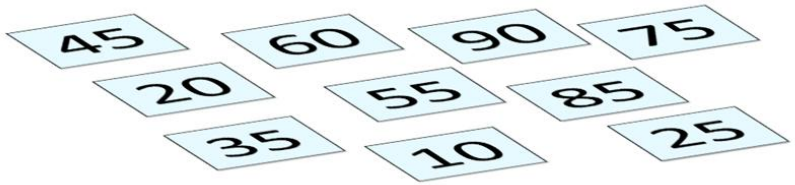
Print(f"")에서 f는 formatted string literal 의미
문자열 앞에 f를 붙이면 { } 안에 파이썬
변수나 식을 직접 넣어 문자열 포맷 간결 표현

2.3 알고리즘의 표현 방법

- 알고리즘의 형태는 단계별 절차이므로, 마치 요리책의 요리를 만드는 절차와 유사
- 알고리즘의 각 단계는 보통 말로 서술할 수 있으며, 컴퓨터 프로그래밍 언어로만 표현할 필요는 없음
- 일반적으로 알고리즘은 프로그래밍 언어와 유사한 의사 코드(pseudo code)로 표현

최대 숫자 찾기 문제를 위한 알고리즘

- 최대 숫자 찾기



- 카드의 숫자를 하나씩 비교하면서 본 숫자들 중에서 가장 큰 숫자를 기억해가며 찾는다.
- 마지막 카드의 숫자를 본 후에, 머릿속에 기억된 가장 큰 숫자가 적힌 카드를 바닥에서 집어 든다.

보통 말로 표현된 알고리즘

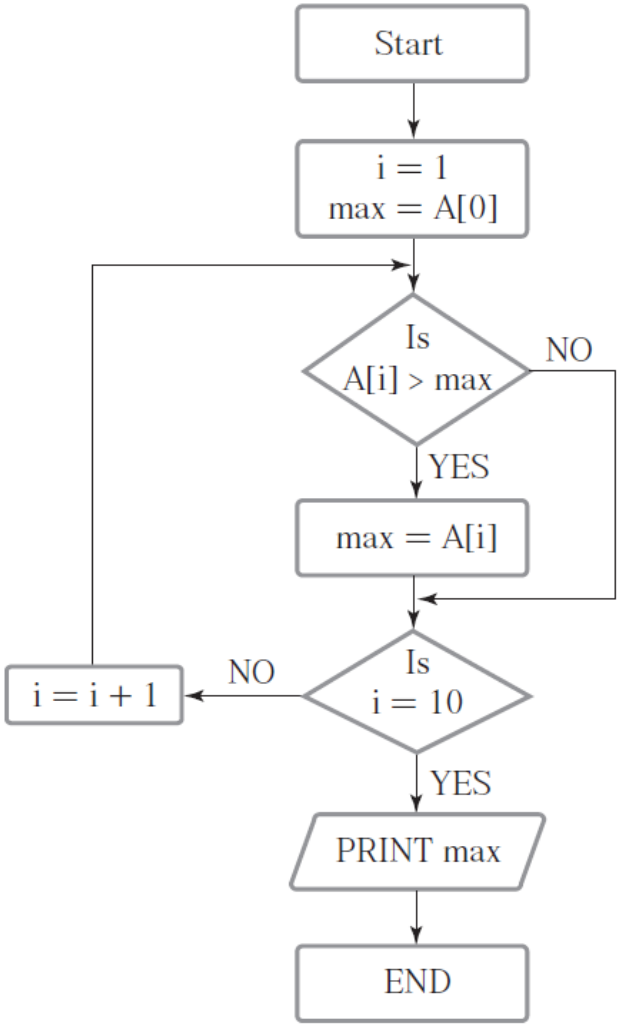
1. 첫 카드의 숫자를 읽고 머릿속에 기억해 둔다.
2. 다음 카드의 숫자를 읽고, 그 숫자를 머릿속의 숫자와 비교한다.
3. 비교 후 큰 숫자를 머릿속에 기억해 둔다.
4. 다음에 읽을 카드가 남아있으면 line 2로 간다.
5. 머릿속에 기억된 숫자가 최대 숫자이다.

의사 코드로 표현된 알고리즘

배열 A 에 10개의 숫자가 있다면

```
max = A[0]
for i = 1 to 9
if (A[i] > max) max = A[i]
return max
```

플로우 차트(flow chart)



최대 숫자 찾기 문제 알고리즘

➢ Python 프로그램

```
def find_max_with_loop(numbers):
    # 빈 리스트가 들어올 경우를 대비한 예외 처리
    if not numbers:
        return "배열이 비어있습니다."
    # 1. 배열의 첫 번째 숫자를 임시 최댓값으로 설정합니다.
    max_num = numbers[0]
    # 2. 배열의 두 번째 숫자부터 끝까지 순회하며 하나씩 확인합니다.
    # numbers[1:] 는 첫 번째 요소를 제외한 나머지 리스트를 의미합니다.
    for current_num in numbers[1:]:
        # 3. 현재 숫자가 임시 최댓값보다 크다면?
        if current_num > max_num:
            # 최댓값을 현재 숫자로 갱신합니다.
            max_num = current_num
    # 반복문이 모두 끝나면 최종적으로 가장 큰 수가 max_num에 남아있게 됩니다.
    return max_num
```

```
# 테스트 리스트
my_array = [45, 20, 35, 60, 55, 10, 90, 85, 25, 75]
result = find_max_with_loop(my_array)
print(f"배열 {my_array}의 최댓값: {result}")
```

2.4 알고리즘의 분류

- 문제의 해결 방식에 따른 분류
  - 분할 정복(Divide-and-Conquer) 알고리즘
  - 그리디(Greedy) 알고리즘
  - 동적 계획(Dynamic Programming) 알고리즘
  - 근사(Approximation) 알고리즘
  - 백트래킹(Backtracking) 알고리즘
  - 분기 한정(Branch-and-Bound) 알고리즘

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 분할 정복(Divide-and-Conquer) 알고리즘

- 어려운 문제를 작은 단위로 쪼개어 정복하는 알고리즘 초석
- 대표적인 예로 정렬 알고리즘 중에서 퀵 정렬이나 합병 정렬과 이진 탐색, 선택문제, 고속 푸리에 변환 문제가 있음
- 분할 정복의 3단계 프로세스

1. 분할 (Divide)

원래 문제를 동일한 유형의 더 작은 하위 문제들로 나눔

문제의 크기가 임계값 이하가 될 때까지 반복

2. 정복 & 결합

하위 문제를 재귀적으로 해결하고(Conquer), 하위 문제의 규모가 나눌 수 없는 단위가 되면 탈출 조건을 설정하고 해결함.

그 결과를 결합하여(Combine) 원래 문제의 해답을 도출

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 분할 정복(Divide-and-Conquer) 알고리즘

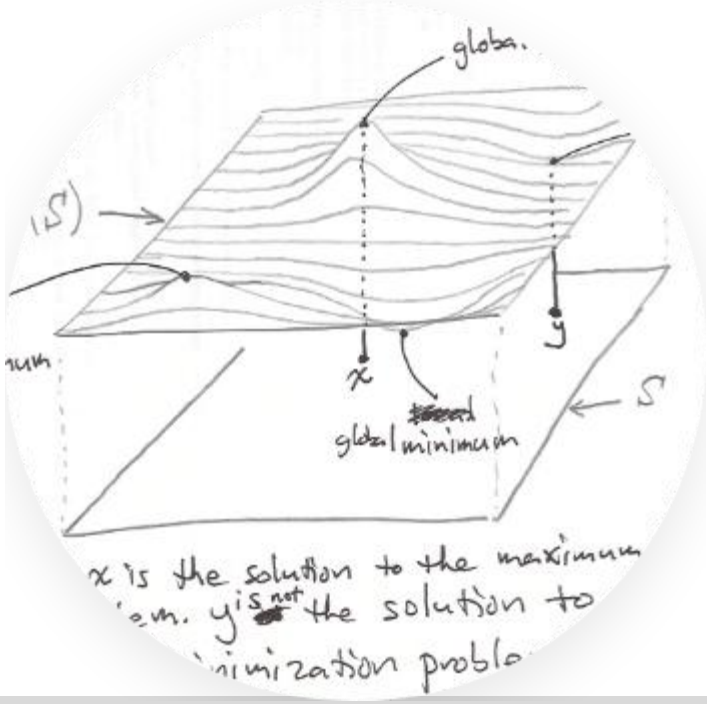
- 분할정복 특징 및 장단점
  - 분할된 작은 문제는 원래 문제와 성격이 동일함 ▶ 입력 크기만 작아짐
  - 분할된 문제는 서로 독립적(중복 제거 없음) ▶ 순환적 분할 및 결합 결합 가능

장점	단점
• Top-down 재귀방식으로 구현하기 때문에 코드가 직관적 • 문제를 나누어 해결한다는 특징상 병렬적으로 문제를 해결할 수 있음	• 재귀 함수 호출로 오버헤드가 발생할 수 있음 - 과도한 함수 호출로 인한 스택 메모리 사용량 증가 - 스택에 다량의 데이터가 보관되는 경우 오버플로우가 발생할 수 있음 • 중복 계산의 위험 - 피보나치 수열과 같이 동일한 하위 문제가 반복될 경우 효율성 급락 - 해결법으로 동적 계획법(DP) 패러다임

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 :그리디(Greedy) 알고리즘

- 미래를 생각하지 않고 현재 상태에서 최선의 선택을 내리는 전략
- 그리디 알고리즘의 결정 구조



탐욕적 선택 속성 (Greedy Choice)

- 지역적인 최적해(Local Optimum)가 결국 전체의 최적해 (Global Optimum)가 될 수 있는 문제에 적용
- 결정 과정에서 이전에 내린 선택을 번복하지 않는 것이 특징

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 :그리디(Greedy) 알고리즘

- 대표 사례 : 허프만 코딩(Huffman)

데이터 압축

문자의 빈도수에 따라 가변 길이 코드를 부여하여 전체 용량을 최소화

트리 구성

빈도수가 가장 낮은 두 노드를 합쳐 나가는 그리디 방식을 사용

최적성 보장

접두부 코드(Prefix Code)의 특성을 유지하며 항상 최적의 압축률을 보장

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 :그리디(Greedy) 알고리즘

- 장점
  - 단순성과 직관성
    - 그리디 알고리즘은 문제를 해결하는 데 있어 간단하고 직관적인 방법을 제공
    - 복잡한 논리나 많은 계산 없이도 문제를 해결할 수 있음
  - 빠른 실행 시간
    - 일반적으로 그리디 알고리즘은 각 단계에서 최적의 선택을 하기 때문에, 비교적 적은 계산과 반복으로 문제를 해결할 수 있음
    - 다른 복잡한 알고리즘에 비해 실행 시간이 짧은 편
  - 적은 메모리 사용
    - 그리디 알고리즘은 보통 현재의 선택만을 고려하므로, 메모리 사용량이 상대적으로 적음
    - 동적 계획법이나 분할 정복법처럼 많은 중간 상태를 ~~저장할 필요~~가 없음
  - 부분 문제 독립성
    - 각 단계에서의 선택이 이후의 선택에 영향을 미치지 않기 때문에, 부분 문제를 독립적으로 해결할 수 있음
    - 이로 인해 병렬 처리나 분산 계산에 유리할 수 있음

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 :그리디(Greedy) 알고리즘

- 단점
  - 최적해 보장 불가
    - 그리디 알고리즘은 항상 최적의 해를 보장하지 않음
    - 특정 문제에서는 서브 최적해(부분적으로 최적화된 해)만을 제공할 수 있음
  - 제약 조건의 중요성
    - 그리디 알고리즘이 올바르게 작동하려면 문제의 특정 조건이 충족되어야 함
    - 이러한 조건이 만족되지 않는 문제에서는 그리디 알고리즘이 적절하지 않을 수 있음
  - 복잡한 문제 해결 불가
    - 복잡한 최적화 문제나 다수의 제약 조건이 있는 문제에서는 그리디 알고리즘이 적합하지 않을 수 있음
    - 예를 들어, 배낭 문제나 TSP(Traveling Salesman Problem)와 같은 문제에서는 다른 고급 알고리즘이 필요함
  - 문제의 특성에 의존
    - 그리디 알고리즘은 문제의 특성에 따라 성능이 크게 좌우됨
    - 동일한 문제라도 입력 데이터의 특성에 따라 알고리즘의 성능이 달라질 수 있음

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 동적 계획(Dynamic Programming) 알고리즘

- 복잡한 문제를 작은 문제로 나누고, 그 해답을 저장하여 재사용하는 영리한 전략
- DP의 두가지 구현 방식

Memoization (Top-Down)

- 재귀적으로 문제를 해결하되, 계산된 결과를 메모리에 저장하여 중복 계산을 방지
- 필요한 하위 문제만 호출함

Tabulation (Bottom-Up)

- 가장 작은 하위 문제부터 시작하여 상향식으로 테이블을 채워나감
- 반복문을 사용하며 오버헤드가 적음

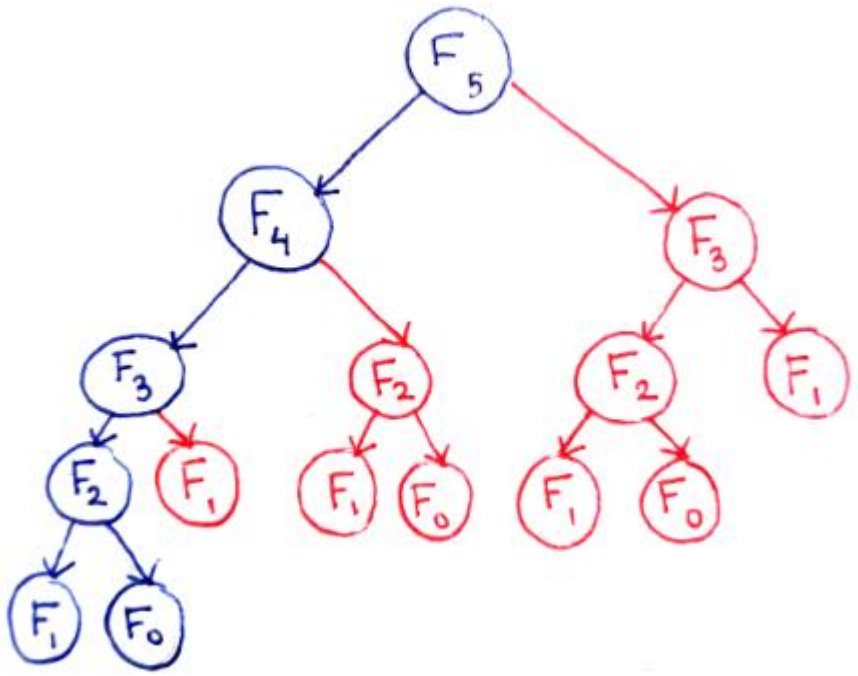
2.4 알고리즘의 분류

➢ **문제의 해결 방식에 따른 분류 : 동적 계획(Dynamic Programming) 알고리즘**

- **중복 부분 문제의 시각화**

Overlapping Subproblems

- **피보나치 수열과 같이 동일한 계산이 여러 번 나타나는 구조에서 DP는 진가를 발휘함**
- **이미 계산된 Fib(n-2)의 결과를 다시 계산하지 않고 즉시 가져옴으로써 지수 시간 복잡도를 다항 시간 복잡도로 혁신적으로 줄임**



2.4 알고리즘의 분류

➢ **문제의 해결 방식에 따른 분류 : 동적 계획(Dynamic Programming) 알고리즘**

- DP 성립을 위한 필수조건

🔑 **최적 부분 구조**: 큰 문제의 최적해가 작은 문제들의 최적해들로부터 구성될 수 있어야 함

🔑 **중복 부분 문제**: 동일한 하위 문제가 여러 번 반복되어야 메모이제이션의 효과가 극대화됨

🔑 **무후성(No After-effect)**: 특정 상태의 해는 그 상태에 도달한 경로와 무관해야 함

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 근사(Approximation) 알고리즘

- 최적해를 찾는 것이 불가능할 때, 다항 시간 내에 '충분히 좋은' 해를 찾는 전략
- 아직 풀리지 않고 있기에 정확한 해를 구하지 못해 최대한 근사한 해를 구하는 방법

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 근사(Approximation) 알고리즘

- 한붓그리기' 게임
  - 게임은 간선과 정점으로 이루어진 그래프가 주어졌을 때, 각각의 간선을 딱 한번씩만 방문해서 모든 정점을 방문 할 수 있는 경로를 찾는 게임
  - 이산수학에서 오일러 회로 (Euler Circle)
  - 한붓그리기 문제를 풀기 위해서는 모든 정점을 시작점으로 잡고 주위에 있는 경로들을 탐색해가며 모든 정점을 방문 할 수 있는지 찾아내야 함
  - 만약 정점의 개수가 적은 숫자라면 쉽게 풀 수 있겠지만, 만약 정점의 개수가 100개라고 생각해보면, 해당 문제를 쉽게 풀 수 없음
  - 만약 우리가 풀고자 하는 문제가 다항식 시간으로 떨어지게 되는 문제를 P(Polynomial) 문제라고 하고, 다항식 시간안에 해결 할 수 없는 한붓그리기 같은 문제를 NP(Non-deterministic Polynomial) 문제

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 근사(Approximation) 알고리즘

- NP-Hard 문제와 현실적 타협

근사 비율 (Approx Ratio)

- 알고리즘이 내놓은 해가 최적해와 얼마나 차이가 나는지를 수학적으로 증명함
- (근사해 / 최적해)로 표현

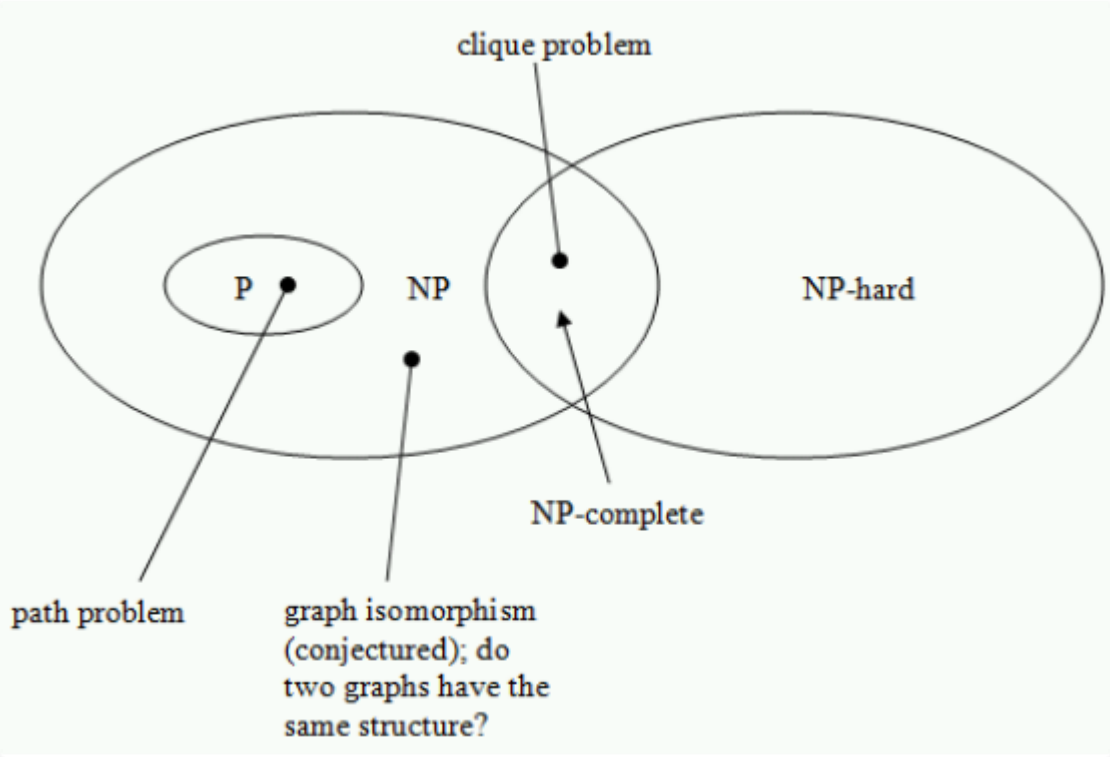
다항 시간 보장

- 최적해를 찾는 데 지수 시간이 걸리는 문제를 다항 시간(Polynomial Time) 내에 해결하여 실무적인 활용성을 높임

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 근사(Approximation) 알고리즘

- 알고리즘 복잡도 클래스 맵



복잡도 계층 구조

- P: 다항 시간에 해결 가능
- NP: 해의 정답 여부를 다항 시간에 확인 가능
- NP-Hard: 가장 어려운 문제군 (근사 알고리즘의 주 무대)
- NP-complete : NP-Hard과 동시에 NP 문제

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 근사(Approximation) 알고리즘

- 장점

높은 효율성(속도)

복잡한 문제를 현실적인 시간 안에 해결할 수 있는 다항 시간 계산을 보장함

보장된 성능

최적해와 근사해 사이의 오차 범위(근사 비율)가 이론적으로 증명된 경우가 많음

실용적 적용

정해진 자원과 시간 제한 내에서 '최선'에 가까운 답이 필요할 때 활용 가능

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 근사(Approximation) 알고리즘

- 단점

최적해 보장 불가

출력된 결과가 최적값(Optimal)이 아닐 수 있음

오차 가능성

알고리즘 설계에 따라 근사해와 실제 최적해 사이의 오차(근사 비율)가 클 수 있음

구현 복잡도

문제에 맞는 근사 알고리즘을 설계하는 과정이 어렵고, 문제의 특징을 깊이 이해해야 함

"시간을 희생하여 완벽한 답을 얻는 대신, 약간의 정확도를 희생하고 빠른 계산 속도를 얻는 방식"

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 백트래킹(Backtracking) 알고리즘

- 모든 가능성을 시도하되, 가능성이 없는 길은 즉시 돌아나오는 탐색 기법
- 상태 공간 트리(State Space Tree)

깊이 우선 탐색 (DFS)

- 해를 찾기 위해 트리의 깊은 부분까지 먼저 탐색
- 재귀 함수를 이용해 간결하게 구현 가능

가지치기 (Pruning)

- 현재 노드가 유망(Promising)하지 않다고 판단되면, 더 이상 자식 노드를 방문하지 않고 부모 노드로 퇴각(Backtrack) 함

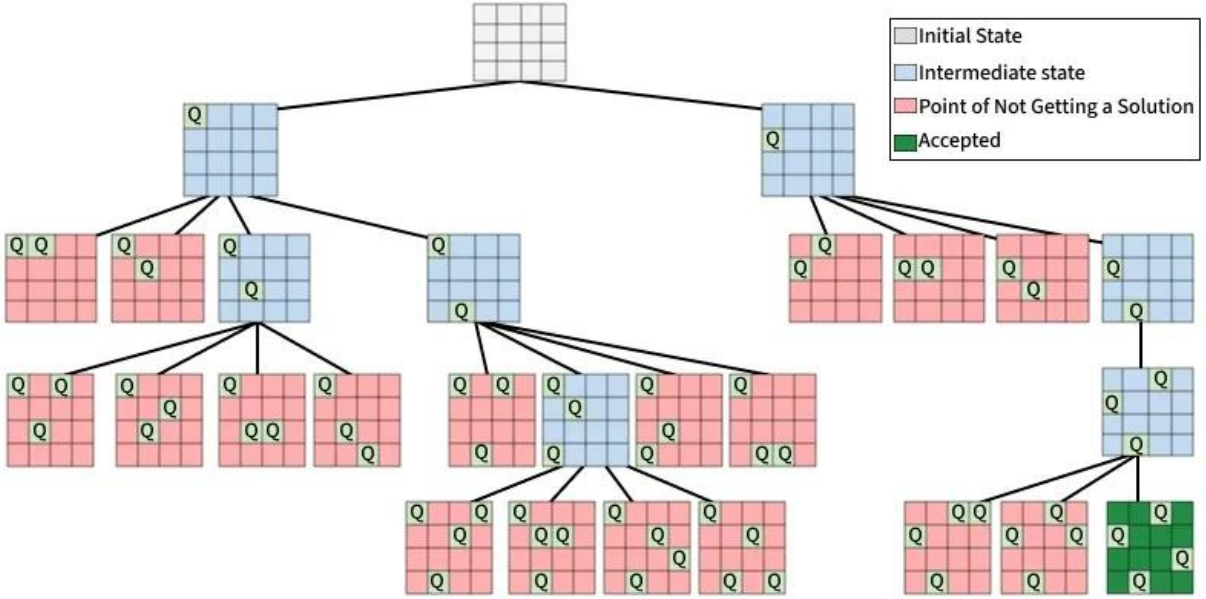
2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 백트래킹(Backtracking) 알고리즘

- N-Queens : 대표적인 백트래킹

N x N 체스판에 N개의 퀸을 서로 공격할 수 없게 배치하는 문제

- 👑 매 행(Row)마다 퀸을 하나씩 배치
- ╲ 같은 열, 대각선 여부 체크
- ↺ 실패 시 이전 행으로 돌아가 위치 수정



2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 백트래킹(Backtracking) 알고리즘

- 장점

탐색 공간 절약

불필요한 경로를 사전에 차단하여 단순 브루트포스보다 훨씬 빠름

모든 해 탐색

단순히 하나의 해가 아니라 조건에 맞는 모든 해를 찾는 데 유리

제약 충족 문제

스도쿠, 미로 찾기 등 복잡한 제약 조건이 있는 문제에 적합

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 분기 한정(Branch-and-Bound) 알고리즘

- 최적화 문제를 해결하기 위해 한정 함수를 사용하는 고급 백트래킹
- B&B의 핵심 : 한정 함수(Bound)

분기 (Branching)

- 문제를 하위 문제로 나누는 과정
- 주로 BFS(너비 우선)나 Best-First Search를 사용

한정 (Bounding)

- 해당 노드 하위에서 얻을 수 있는 해의 한계치(Bound)를 계산하여, 현재 최적해보다 나쁠 경우 탐색을 중단함

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 분기 한정(Branch-and-Bound) 알고리즘

- B&B를 통한 탐색 효율 극대화

70%

탐색 공간 축소율

정교한 한정치의 효과

- 단순한 백트래킹보다 계산량은 많지만, 하한선(Lower Bound) 또는 상한선(Upper Bound)을 이용해 탐색해야 할 노드의 수를 획기적으로 줄여 최적화 문제의 수행 시간을 단축함

2.4 알고리즘의 분류

➢ 문제의 해결 방식에 따른 분류 : 분기 한정(Branch-and-Bound) 알고리즘

- **백트래킹 vs 분기한정**

비교 항목	백트래킹 (Backtracking)	분기 한정 (Branch & Bound)
주요 목적	모든 해 찾기 / 결정 문제	최적해 찾기 (최소/최대)
탐색 방식	DFS (깊이 우선)	BFS / Best-First (우선순위 큐)
핵심 기법	유망성 판단 (Pruning)	한정치 계산 (Bounding)

알고리즘의 분류

- 문제에 기반한 분류
  - 정렬 알고리즘
  - 그래프 알고리즘
  - 기하 알고리즘
- 특정 환경에 따른 분류
  - 병렬(Parallel) 알고리즘
  - 분산(Distributed) 알고리즘
  - 양자(Quantum) 알고리즘
- 기타 알고리즘

알고리즘의 분류

➢ 문제에 기반한 분류 : 정렬 알고리즘(Sorting Algorithms)

- 정렬 알고리즘의 기초 (Selection & Bubble)

선택 정렬 (Selection Sort)

가장 작은 요소를 찾아 맨 앞의 요소와 교체하는 방식

- ✓ 시간 복잡도: $O(n^2)$
- ✓ 데이터 교환 횟수가 적은 장점이 있음

거품 정렬 (Bubble Sort)

인접한 두 요소를 비교하며 큰 값을 뒤로 보내는 방식

- ✓ 시간 복잡도: $O(n^2)$
- ✓ 구현이 매우 단순하나 성능이 낮음

알고리즘의 분류

➢ 문제에 기반한 분류 : 정렬 알고리즘(Sorting Algorithms)

- 분할 정복 정렬(Quick & Merge)

합병 정렬 (Merge Sort)

데이터를 반으로 나누고 각각 정렬 후 합치는 Stable Sort

- ✓ 최악/평균 모두 $O(n \log n)$
- ✓ 추가적인 메모리 공간이 필요

퀵 정렬 (Quick Sort)

피벗을 기준으로 분할하여 정렬하는 In-place Sort

- ✓ 평균 $O(n \log n)$으로 매우 빠름
- ✓ 최악의 경우 $O(n^2)$ 이 발생할 수 있음

알고리즘의 분류

➢ 문제에 기반한 분류 : 정렬 알고리즘(Sorting Algorithms)

- 비비교 정렬(Counting & Radix)
  - 비교 연산 없이 데이터의 특성을 이용하여 $O(n)$ 수준의 속도를 달성

계수 정렬 (Counting)

데이터의 빈도수를 계산하여 정렬. 값의 범위가 좁을 때 극단적으로 빠름

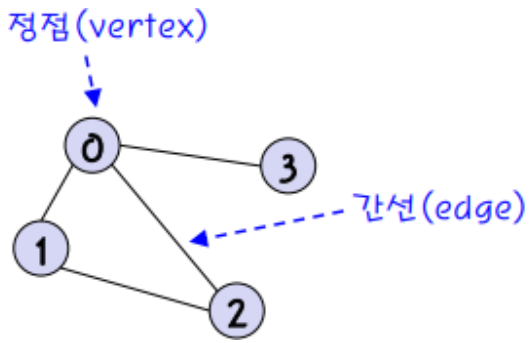
기수 정렬 (Radix)

자릿수별로 정렬을 수행. 큰 정수나 문자열 데이터 처리에 적합함

알고리즘의 분류

➢ 문제에 기반한 분류 :그래프 알고리즘

- 그래프를 표현하고 이해하는 알고리즘
  - 지도가 있고 그 지도에 우리 집, 학교, 친구 집 등이 표현 되어 있음
  - 우리 집에서 학교까지의 거리가 가까운지, 친구 집에서 학교까지가 가까운지 알아보고 싶다면 각 집들과 학교를 어떻게 표현할 지 부터 알아야 함
  - 그 다음엔 우리 집에서 학교까지 갈 수 있는지, 학교에서 친구 집까지는 갈 수 있는지 알아야 함
  - 갈 수 있다면 각각의 최단 경로는 어떻게 되는지, 길의 연결이 끊기면 다른 길은 없는지 등등을 알아보는 것이 그래프 알고리즘
- 여기서 각각의 집이나 학교들은 노드 혹은 정점이라고 표현
- 연결되어 있는 길은 엣지 혹은 간선이라 표현



알고리즘의 분류

➢ 문제에 기반한 분류 :그래프 알고리즘

- 최단 경로 알고리즘
  - 연결이 되었다는 것을 알았다면 다음으로 두 정점의 최단 경로 방법
  - 대표적인 알고리즘으로 다익스트라 알고리즘, 벨만 포드 알고리즘, 플로이드 와샬 알고리즘
- 최소 신장 트리
  - 모든 정점을 연결하는 최단 경로 방법
  - 각각의 거리는 최단 경로가 아닐 수도 있지만 모든 경로를 지나는 최소값을 구하는 것
  - 대표적인 알고리즘으로 쿠르스칼 알고리즘, 프림 알고리즘

알고리즘의 분류

➢ 문제에 기반한 분류 :그래프 알고리즘

- 그래프 탐색 알고리즘(BFS & DFS)

너비 우선 탐색 (BFS, Breadth Fisrt Search)

큐(Queue)를 사용하여 인접 노드를 먼저 방문

- ✓ 최단 경로 탐색에 유리
- ✓ 가중치가 없는 그래프의 최단거리 보장

깊이 우선 탐색 (DFS, Depth First Search)

스택(Stack)이나 재귀를 사용하여 끝까지 탐색

- ✓ 모든 노드 방문 및 사이클 감지
- ✓ 백트래킹 및 위상 정렬에 활용

알고리즘의 분류

- 문제에 기반한 분류 :그래프 알고리즘

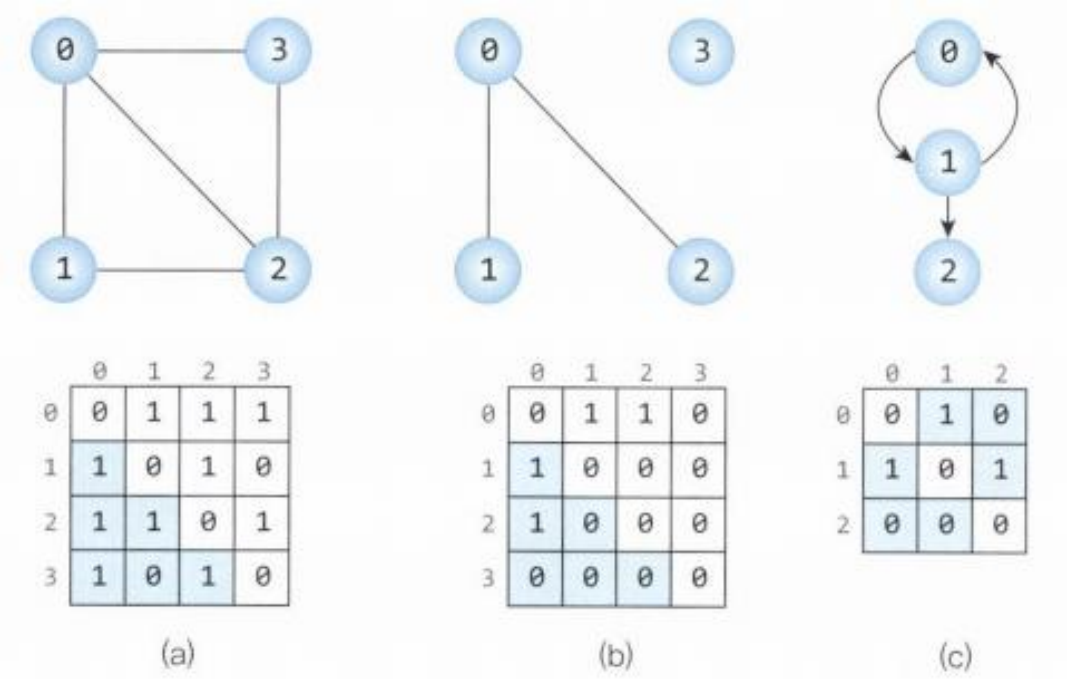
  - 그래프 형태
    - 그래프 형태도 크게 두가지가 존재
    - 인접 행렬은 이차원 배열 형태의 그래프
    - 인접 리스트는 노드와 간선의 연결로 표현한 방식

알고리즘의 분류

➢ 문제에 기반한 분류 :그래프 알고리즘

- 그래프 형태 : 인접 행렬

- 그래프의 정점을 2차원 배열로 작성
- 정점의 개수가 n이라면 n*n 형태의 2차원 배열이 인접 행렬로 사용
- 인접 행렬에서 행과 열은 정점을 의미
- 각의 원소들은 정점 간의 간선 표시



- 장점
  - 2차원 배열에 모든 정점들의 간선 정보가 있기 때문에,
    두 정점을 연결하는 간선을 조회할 때 O(1) 시간복잡도로 가능
  - 정점(i)의 차수를 구할 때는 다음과 같이 인접행렬(M)의 i번째
    행의 값을 모두 더하면 되므로 O(n)의 시간복잡도를 가짐
  - 구현이 비교적 간단하다.

- 단점
  - 간선의 수와 무관하게 항상 n^2 크기의 2차원 배열이 필요하므로 메모리 공간이 낭비
  - 그래프의 모든 간선의 수를 알아내려면 인접행렬 전체를 확인해야 하므로 $O(n^2)$의 시간이 소요

알고리즘의 분류

➢ 문제에 기반한 분류 :그래프 알고리즘

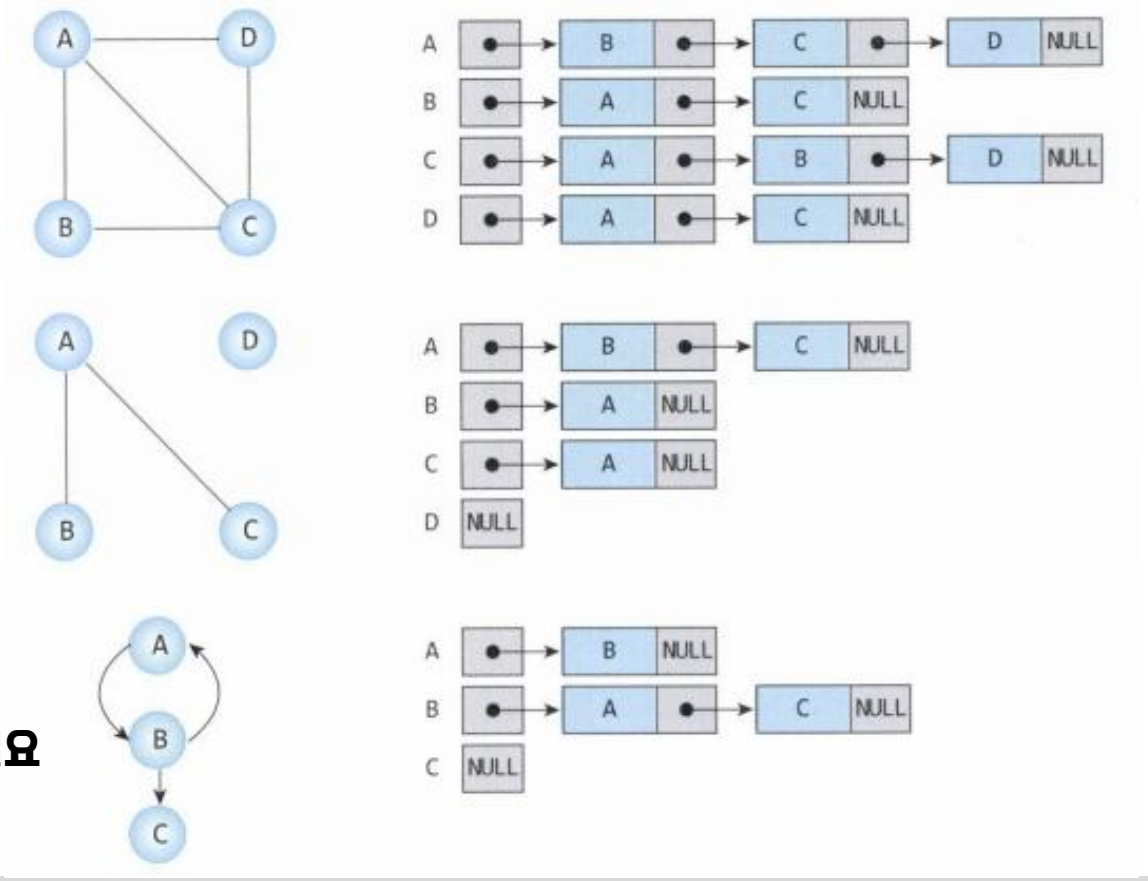
- 그래프 형태 : 인접 리스트

- 그래프의 각 정점에 인접한 정점들을 연결리스트(Linked List)로 표현

- 정점의 개수만큼 인접리스트가 존재하며, 각각의 인접리스트에는 인접한
- 정점 정보가 저장
- 그래프는 각 인접리스트에 대한 헤드포인터를 배열

- 장점
  - 존재하는 간선만 관리하면 되므로 메모리 사용 측면에서 보다 효율적
  - 그래프의 모든 간선의 수를 알아내려면 각 정점의 헤더 노드부터 모든 인접리스트를 탐색해야 하므로 O(n+e)의 시간이 소요

- 단점
  - 두 정점을 연결하는 간선을 조회하거나 정점의 차수를 알기 위해서는 정점의 인접 리스트를 탐색해야 하므로 정점의 차수만큼의 시간이 필요
  - 구현이 비교적 어렵다.



알고리즘의 분류

- **문제에 기반한 분류 : 기하 알고리즘**
  - 기하학의 기본요소로는 점, 선, 다각형, 다면체
  - 기하문제를 풀기 위해서는 기본요소인 점, 선, 다각형, 다면체 등을 어떻게 표현할 것인가를 결정해야하는데, 간단한 표현법으로 좌표를 사용

알고리즘의 분류

문제에 기반한 분류 : 기하 알고리즘

- 점, 벡터, CCW

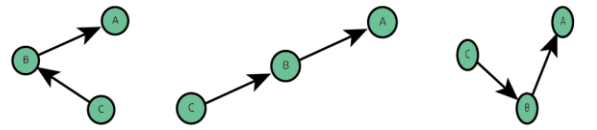
기본 요소: 모든 기하 알고리즘은 2D/3D 공간의 점(Point)과 그 사이의 방향을 나타내는 벡터(Vector)에서 시작

방향 판별 (CCW): 세 점의 위치 관계를 파악하는 핵심 알고리즘으로, 벡터의 외적을 활용하여 계산

계산 원리: CCW 알고리즘은 아래 수식을 통해 판별

- CCW(Counter Clock Wise) 알고리즘은 3개의 점 A, B, C가 있을 때 이 점 3개를 이은 직선의 방향을 알고자 할 때 유용한 기하 알고리즘
- 경우의 수는 총 3가지가 있으며 시계, 반시계, 직선이 있으며 이는 외적을 통하여 구할 수 있으며 외적의 결과가 음수이면 시계 방향, 직선은 0, 반시계 방향일 경우 양수

$$CCW(A, B, C) = (x_1 y_2 + x_2 y_3 + x_3 y_1) - (y_1 x_2 + y_2 x_3 + y_3 x_1)$$

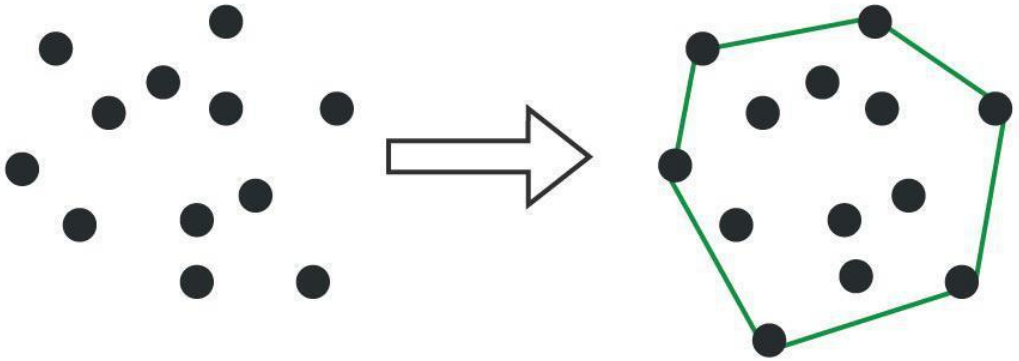


알고리즘의 분류

문제에 기반한 분류 : 기하 알고리즘

- 블록 껍질(Convex Hull)

평면 위에 흩어진 점들을 모두 포함하는 가장 작은 볼록 다각형을 만드는 알고리즘



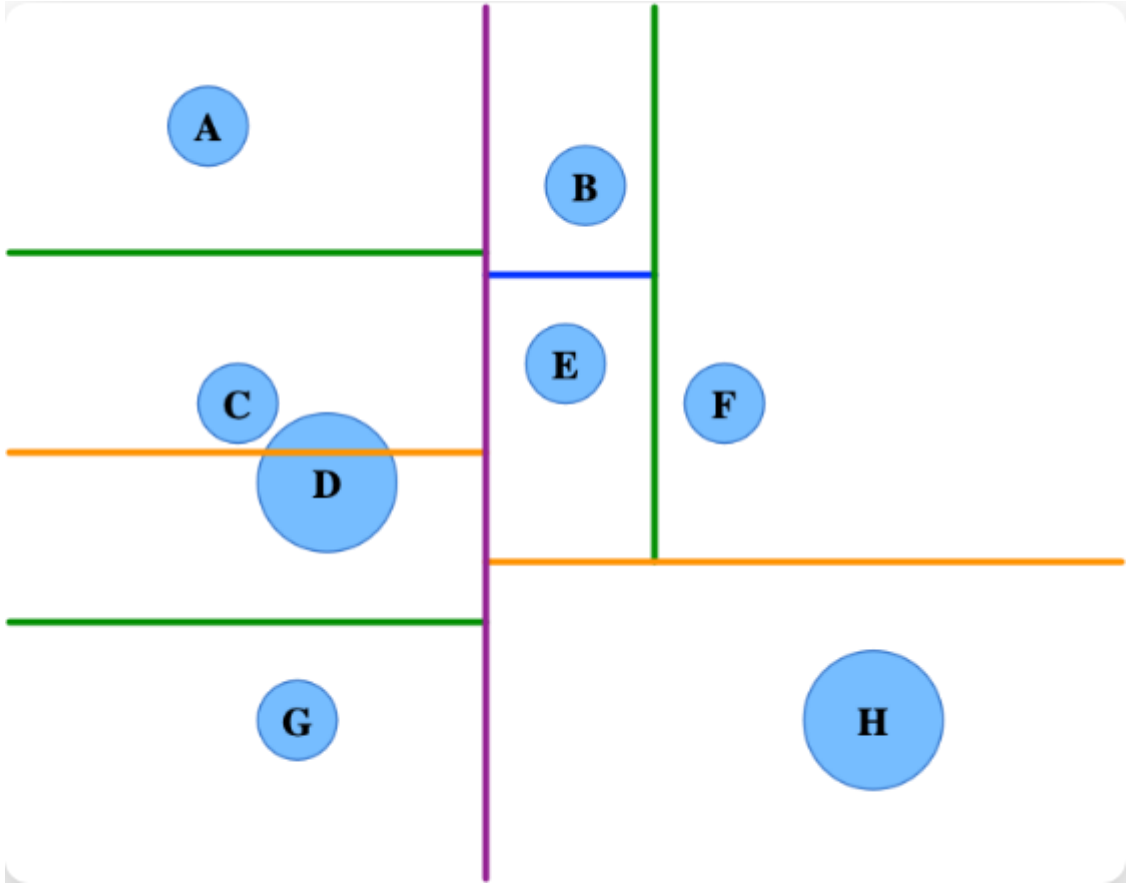
- 선물 포장 알고리즘 (Gift Wrapping)
- 그레이엄 스캔 (Graham Scan)
- 단조 체인 (Monotone Chain)

알고리즘의 분류

- 문제에 기반한 분류 : 기하 알고리즘
  - 공간 분할과 탐색 : 복잡한 데이터를 구조화

KD-트리는 k차원의 공간을 이분화하여 점들을 저장하는 이진 트리 데이터 구조

- 순차적 분할: 각 레벨마다 x축, y축을 번갈아 가며 중앙값을 기준으로 공간을 나눔
- 성능: 범위 검색(Range Search)과 최근접 이웃 탐색(NN Search)에서 탁월한 성능



알고리즘의 분류

➢ 문제에 기반한 분류 : 기하 알고리즘

- 주요 활용

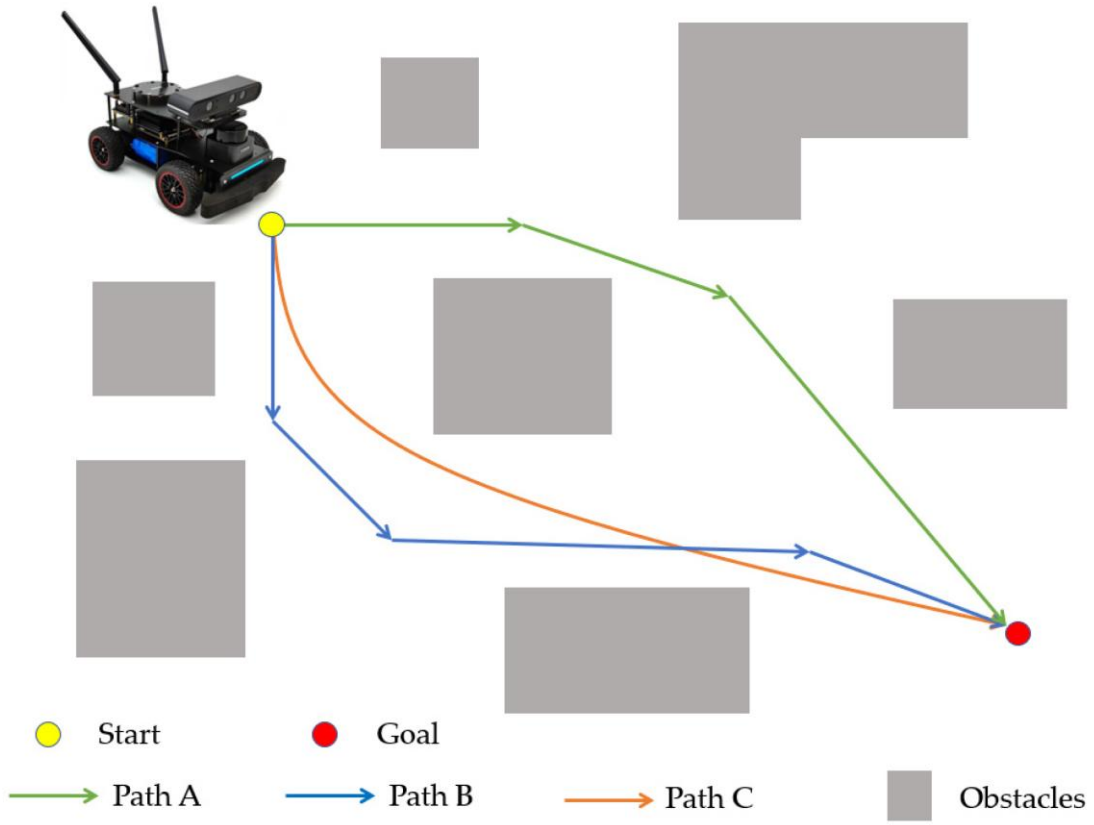
산업 분야	핵심 알고리즘	실제 활용 사례
GIS / 지도	Point in Polygon	사용자 위치 기반 행정 구역 판별
로보틱스	Path Planning	자율 주행 로봇의 최적 경로 탐색
게임 엔진	Collision Detection	실시간 객체 간 충돌 및 물리 엔진
컴퓨터 그래픽스	Triangulation	3D 모델의 표면 렌더링 및 메쉬 생성

알고리즘의 분류

- **문제에 기반한 분류 : 기하 알고리즘**
  - 주요 활용 : 로보틱스와 경로계획

기하 알고리즘은 로봇이 장애물 사이에서 충돌 없이 목적지까지 도달하는 길을 찾는 데 필수적

- 가시성 그래프(Visibility Graph)나 구성 공간(Configuration Space) 기법을 통해 로봇의 물리적 크기를 고려한 정밀한 제어가 가능



알고리즘의 분류

- 특정 환경에 따른 분류 : 병렬(Parallel) 알고리즘
  - 직렬처리와 병렬처리의 차이

Serial (직렬)

단일 명령 스트림을 하나씩 순차적으로 실행

알고리즘의 실행 시간이 문제 크기에 비례하여 증가하며, CPU 클럭 속도 향상에 의존

Parallel (병렬)

문제를 여러 개의 독립적인 태스크로 나누어 여러 프로세서에서 동시에 실행

리소스를 효율적으로 활용하여 계산 시간을 단축

알고리즘의 분류

- 특정 환경에 따른 분류 : 병렬(Parallel) 알고리즘
  - 암달의 법칙 (Amdahl's Law)

95%

병렬화 가능 비율 가정 시

알고리즘에서 병렬화할 수 없는 순차 부분(Sequential Fraction)이

전체 실행 속도의 한계를 결정한다는 법칙

어떤 작업의 시간 효율을 개선할 때, 전체 작업시간에 대해 P만큼의 작업시간을 차지하는 작업의 효율을 S만큼 향상시켰다 가정하면

전체 작업 효율은 다음과 같이 향상

$$\frac{1}{(1-P)+\frac{P}{S}}$$

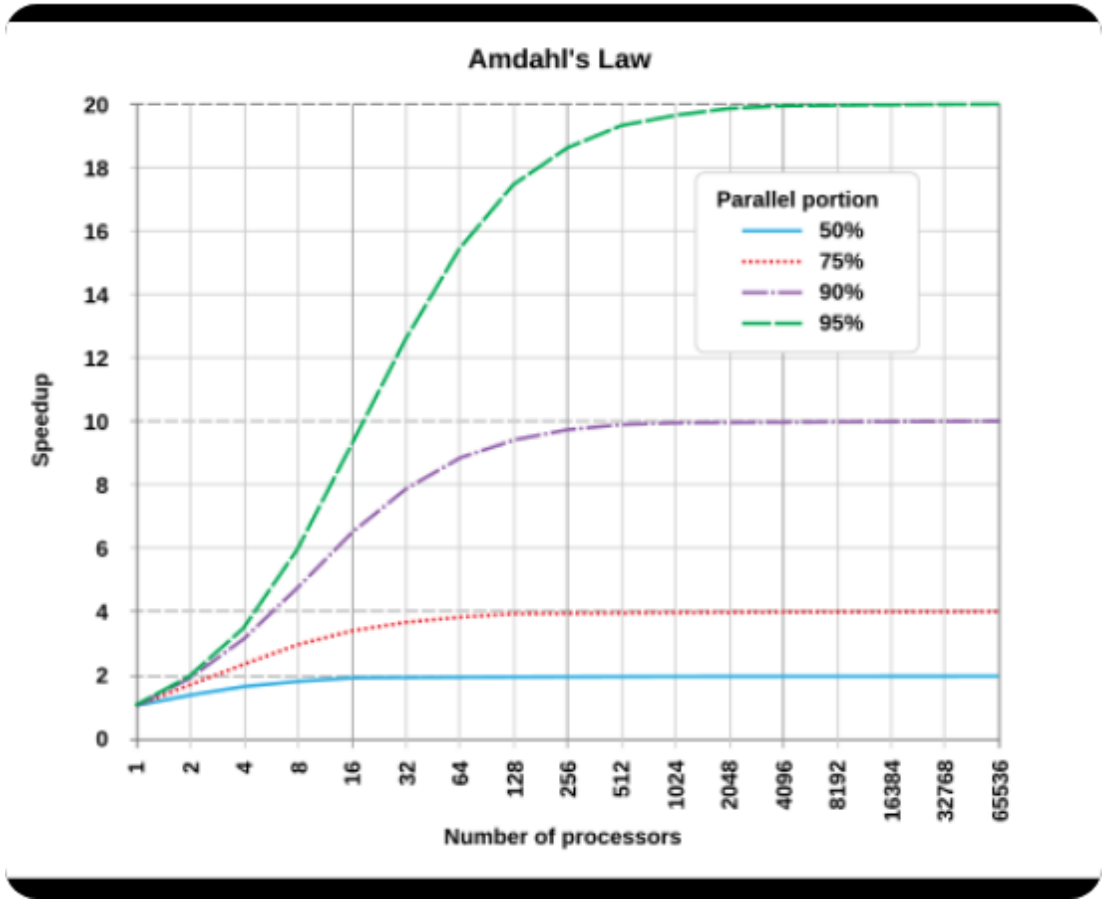
알고리즘의 분류

- **특정 환경에 따른 분류 : 병렬(Parallel) 알고리즘**
  - 대형 문제를 여러 개의 작은 작업으로 분할하여 다수의 프로세서에서 동시에 처리함으로써 계산 속도를 높이는 방식
  - 순차 알고리즘과 달리 병렬 프로그래밍 기술을 활용해 데이터 세트가 커질수록 효율적이며, 딥러닝 경량화나 대규모 데이터 분석 등에서 높은 확장성을 제공함
  - 병렬 알고리즘의 주요 기법
    - 데이터 병렬성 : 동일한 작업을 여러 데이터에 동시에 적용
    - 작업 병렬성 : 여러 작업을 동시에 수행
    - 동기화 : 작업 간의 일관성을 유지하기 위해 동기화 메커니즘 사용

알고리즘의 분류

➢ 특정 환경에 따른 분류 : 병렬(Parallel) 알고리즘

- 성능 곡선의 이해



프로세서 수를 늘린다고 해서 속도가 무한정 빨라지지는 않음

- ✓ Diminishing Returns(수익 감소) : 오버헤드 증가로 인한 효율 저하
- ✓ Gustafson's Law : 데이터 크기가 커지면 더 높은 효율 달성 가능

알고리즘의 분류

➢ 특정 환경에 따른 분류 : 병렬(Parallel) 알고리즘

- PCAM : 알고리즘 설계 4단계

Partitioning

데이터나 계산을 잘게 나누어 태스크로 분해

Communication

태스크 간 정보 교환에 필요한 통신 구조를 정의

Aggregation

작은 태스크들을 묶어 통신 오버헤드를 줄임

Mapping

프로세서에 태스크를 효율적으로 할당

알고리즘의 분류

➢ 특정 환경에 따른 분류 : 병렬(Parallel) 알고리즘

- 병렬 알고리즘 활용 분야 : 그래프 및 빅데이터 처리

현대의 병렬 알고리즘은 거대 그래프 탐색 (BFS/DFS)과 대규모 데이터 분석의 핵심

- MapReduce : 분산 처리를 위한 핵심 알고리즘 패턴
- Pregel : 정점 중심의 대규모 그래프 병렬 처리
- MPI : HPC(고성능 컴퓨팅) 표준 통신 인터페이스

알고리즘의 분류

- 특정 환경에 따른 분류 : 분산(Distributed) 알고리즘

알고리즘의 분류

➢ 특정 환경에 따른 분류 : 양자(Quantum) 알고리즘

- 양자 역학의 핵심 원리

중첩 (Superposition)

큐비트가 0과 1의 상태를 동시에 가질 수 있는 현상
이는 양자 알고리즘이 병렬적으로 정보를 처리할 수 있는 기초가 됨

얽힘 (Entanglement)

두 큐비트가 강하게 연결되어 한 쪽의 상태가 다른 쪽의 상태를 즉각적으로 결정하는 현상
데이터 전송과 복합 계산에 필수적

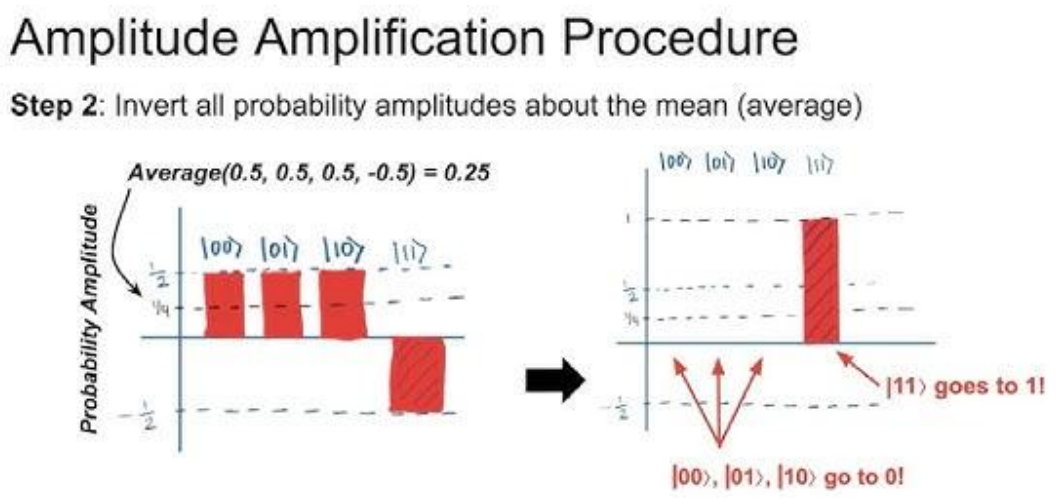
알고리즘의 분류

➢ **특정 환경에 따른 분류 : 양자(Quantum) 알고리즘**

- **양자 알고리즘 : Grover's 알고리즘**

- **비정렬 데이터베이스 검색에서** O(√N)**의 속도를 제공**

- **고전적인 검색 방식이 평균 N/2번의 탐색이 필요한 반면, 그로버 알고리즘은 확률 진폭을 증폭시켜 정답을 찾아내는 획기적인 이차적 가속을 실현**



알고리즘의 분류

- 특정 환경에 따른 분류 : 양자(Quantum) 알고리즘

  - 양자 알고리즘 : Shor's 알고리즘(소인수 분해)

106배+

고전 알고리즘 대비 가속도

- 쇼어 알고리즘은 거대한 정수의 소인수 분해를 다항 시간 내에 해결
- 이는 현재 뱅킹 및 통신에 사용되는 RSA 암호 체계를 무력화할 수 있는 잠재력을 가지고 있어, 포스트 양자 암호화 연구의 기폭제가 됨

알고리즘의 분류

➢ 특정 환경에 따른 분류 : 양자(Quantum) 알고리즘

- 양자 알고리즘의 활용 분야

보안 및 암호학

양자 내성 암호(PQC) 개발 및 절대적인 보안을 제공하는 양자 키 분배(QKD).

신약 및 신소재

분자 구조를 완벽하게 시뮬레이션하여 개발 기간을 수십 년에서 며칠로 단축.

최적화 문제

물류, 금융 포트폴리오 최적화 등 복잡한 변수 사이의 최적해 도출.

알고리즘의 분류

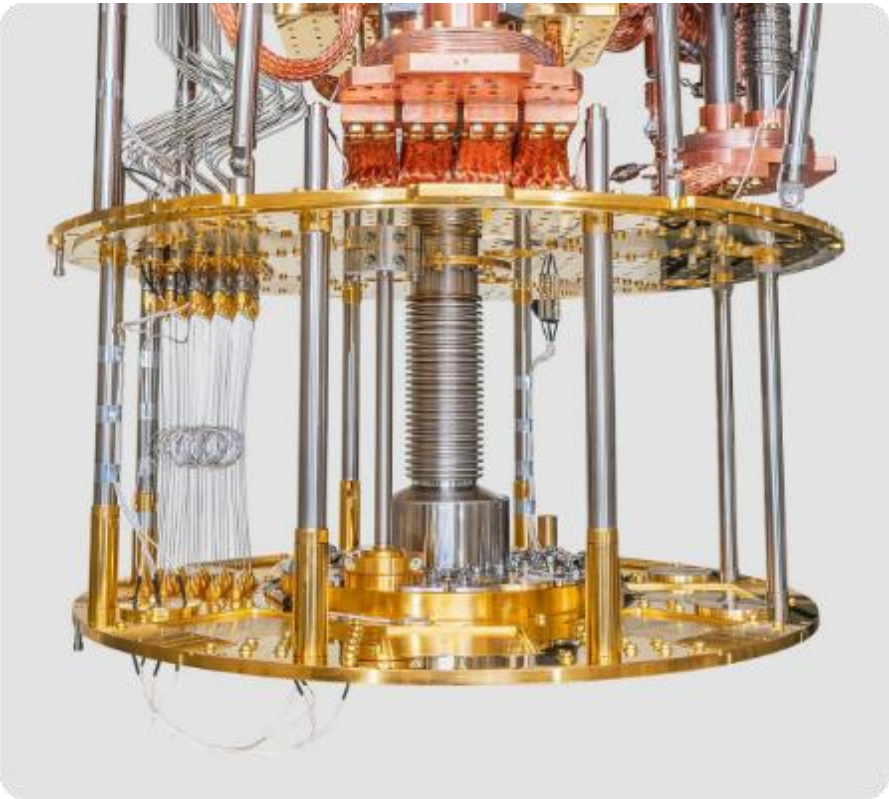
➢ 특정 환경에 따른 분류 : 양자(Quantum) 알고리즘

- 양자 알고리즘 해결 과제

✦ **결어긋남 (Decoherence):** 외부 환경과의 상호작용으로 인해 양자 상태가 소실되는 현상.

✦ **오류 수정 (Error Correction):** 큐비트의 불안정성을 보완하기 위한 고도의 결함 허용 기술 필요.

✦ **냉각 시스템:** 절대 영도에 가까운 온도를 유지하기 위한 거대 설비 필요.



2.5 알고리즘의 효율성 표현

- 알고리즘의 효율성
  - 알고리즘의 수행 시간 또는 알고리즘이 수행하는 동안 사용되는 메모리 크기로 나타낼 수 있음
  - 시간 복잡도(time complexity), 공간 복잡도(space complexity)
  - 일반적으로 알고리즘들을 비교할 때에는 시간 복잡도가 주로 사용됨

시간 복잡도

- 시간 복잡도는 알고리즘이 실행되는 동안에 사용된 기본적인 연산 횟수를 입력 크기의 함수로 나타냄
  - 기본 연산(Elementary Operation)이란 데이터 간 크기 비교, 데이터 읽기, 갱신, 숫자 계산 등과 같은 단순한 연산을 의미

- [예] 10장의 숫자 카드들 중에서 최대 숫자 찾기
  - 순차 탐색으로 찾는 경우에 숫자 비교가 기본적인 연산이고, 총 비교 횟수는 9번
  - n장의 카드가 있다면, (n-1)번의 비교 수행으로 시간 복잡도는 (n-1)

알고리즘의 복잡도 표현 방법

- 최악 경우 분석(Worst-case Analysis)
  - '어떤 입력이 주어지더라도 알고리즘의 수행시간이 얼마 이상은 넘지 않는다'라는 상한(Upper Bound)의 의미
- 평균 경우 분석(Average-case Analysis)
  - 입력의 확률 분포를 가정하여 분석하는데, 일반적으로 균등 분포(Uniform Distribution)를 가정
- 최선 경우 분석(Best-case Analysis)
  - 가장 빠른 수행 시간을 분석하며, 최적(Optimal) 알고리즘을 찾는데 활용

알고리즘의 복잡도 표현 방법

- 상각 분석(Amortized Analysis)
  - 일련의 연산을 수행하여 총 수행 시간을 합하고 이를 연산 횟수로 나누어 수행 시간을 분석
  - [조건] 알고리즘에서 적어도 두 종류의 연산이 수행되고, 그 중 하나는 수행 시간이 길고, 다른 하나는 짧으며, 수행 시간이 짧은 연산은 많이 수행되고 수행 시간이 긴 연산은 적게 수행되어야 상각 분석이 의미를 갖음
  - amortize는 '분할 상환하다'라는 뜻을 가짐. 그리고 상각(償却)은 '보상하여 갚아주다'라는 뜻
- 일반적으로 알고리즘의 수행 시간은 최악 경우 분석으로 표현

등교 시간 분석

- 집에서 지하철역까지 6분, 지하철을 타면 학교까지 20분, 강의실까지 걸어서 10분 걸린다.

- 최선 경우
  - 집을 나와서 6분 후 지하철역에 도착하고, 운이 좋게 바로 열차를 탄 경우를 의미
  - 최선 경우 시간은 6 + 20 + 10 = 36분

6분

20분

10분

등교 시간 분석

- 최악 경우
  - 열차에 승차하려는 순간, 열차의 문이 닫혀서 다음 열차를 기다려야 하고 다음 열차가 4분 후에 도착한다면, 최악 경우는 6 + 4 + 20 + 10 = 40분

등교 시간 분석

- 평균 경우
  - 대략 최악과 최선의 중간이라고 가정했을 때, 38분

등교 시간 분석

- 상각 분석
  - 단순히 1회의 등교 시간을 분석하는 것이 아니라, 예를 들어, 한 학기 동안의 등교 시간을 분석해본다는 점에서 그 의미를 가짐
  - 한 학기 동안 100번 학교에 갔는데 대부분은 지하철을 이용하지만 실제로 지하철보다 시간이 오래 걸리지만 버스를 타고 가끔 학교에 갈 때도 있다면
  - 최악 경우 분석은 버스를 타고 등교하는 시간이 60분이라면 60×100 = 6,000분을 넘지 않는 것으로 표현
  - 상각 분석은 한 학기 동안 학교에 가는데 소요된 시간을 모두 합해서 학교에 간 횟수인 100으로 나눈 값을 1회 등교 시간으로 분석
  - 대표적인 분석 예제: 크러스컬(Kruskal)의 최소 신장 트리 알고리즘

2.6 복잡도의 점근적 표기

➢ 알고리즈의 효율성의 중요성

하드웨어 성능은 매년 비약적으로 발전하지만, 처리해야 할 데이터의 양(Data Volume)은 그보다 훨씬 빠르게 증가

- 하드웨어는 유한하지만 알고리즘은 무한
- 효율적인 알고리즘은 불가능한 계산을 가능
- 입력 크기 n이 커질 때 알고리즘이 어떻게 반응하는지가 분석의 핵심

2.6 복잡도의 점근적 표기

- 시간 복잡도는 입력 크기에 대한 함수로 표기
  - 함수는 여러 개의 항을 가지는 다항식
  - 이를 입력의 크기에 대한 함수로 표현하기 위해 점근적 표기(Asymptotic Notation)를 사용

2.6 복잡도의 점근적 표기

➢ 점근적 분석이란 무엇인가?

점근적 의미

입력 크기 n이 무한대에 가까워질 때 함수의 거동을 분석하는 것을 의미

환경 독립성

OS, CPU 속도, 프로그래밍 언어 등 외부 환경을 배제하고 알고리즘 자체의 논리적 구조를 평가

증가율 중심

정확한 실행 시간이 아닌, 데이터 증가에 따른 '비용의 증가 속도'에 집중

2.6 복잡도의 점근적 표기

➢ 우리가 무시하는 요소들

$f(n) = 3n^2 + 10n + 5$ 함수 에서

$O(n^2)$

추상화의 힘

점근적 분석에서는 다음 두 요소를 과감히 생략함

- 상수 계수: n이 커지면 3이나 1000이나 영향력이 미미해짐
- 낮은 차수의 항: n^2 항에 비해 n 항은 무시할 수 있는 수준이 됨

2.6 복잡도의 점근적 표기

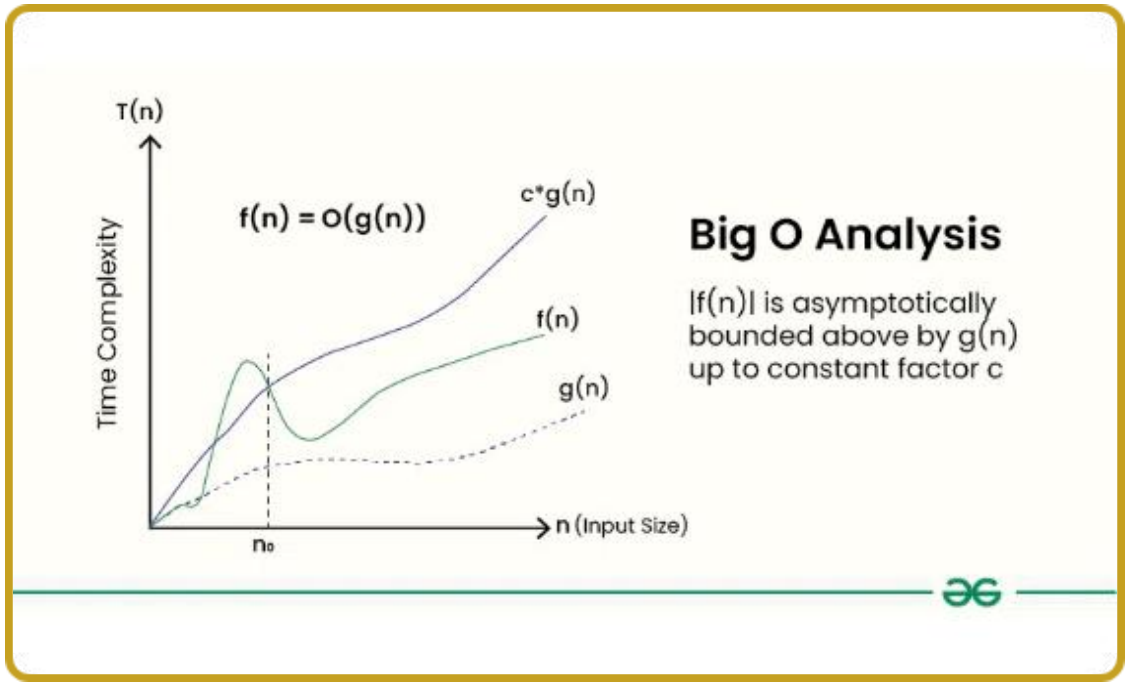
- 점근적 표기
  - 입력 크기 n이 무한대로 커질 때의 복잡도를 간단히 표현하기 위해 사용하는 표기법
    - (Big-Oh)-표기
    - (Big-Omega)-표기
    - (Theta)-표기

O(Big-Oh)-표기

점근적 상한 (Upper Bound)

- 알고리즘의 실행 시간이 "아무리 나빠도 이보다는 좋다"는 것을 의미
- 일반적으로 알고리즘의 최악의 경우(Worst Case)를 표현하는 데 사용됨

$$f(n) = O(g(n))$$

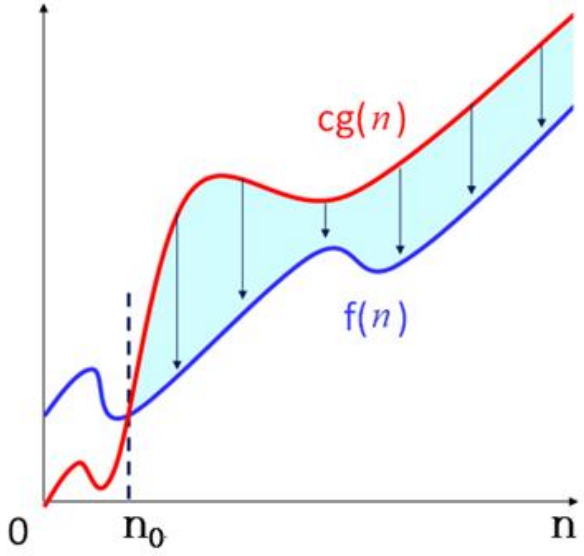


O(Big-Oh)-표기

- O-표기의 정의
  - 모든 $n \geq n_0$에 대해서 $f(n) \leq cg(n)$이 성립하는 양의 상수 c와 n_0가 존재하면, $f(n) = O(g(n))$
- O-표기의 의미
  - n_0와 같거나 큰 모든 n (즉, n_0 이후의 모든 n)에 대해서 f(n)이 cg(n)보다 크지 않다는 것
- f(n) = O(g(n))은 n_0 보다 큰 모든 n 대해서 f(n)이 양의 상수를 곱한 g(n)에 미치지 못한다는 뜻
- g(n)을 f(n)의 상한(Upper Bound)이라 함

O(Big-Oh)-표기

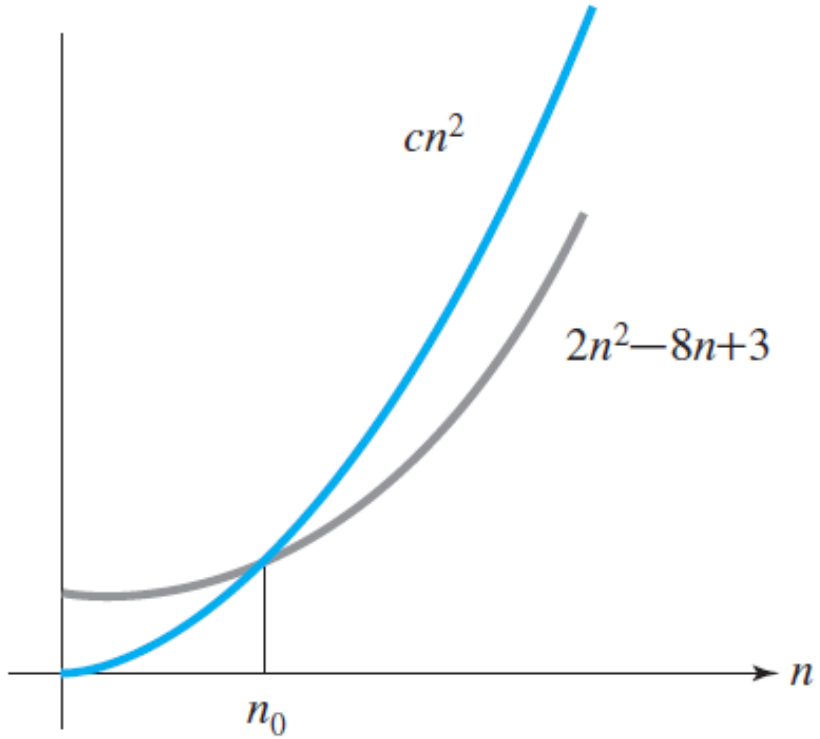
- f(n) = O(g(n))
  - n이 증가함에 따라 O(g(n))이 점근적 상한이라는 것 (즉, g(n)이 n_0보다 큰 모든 n에 대해서 항상 f(n)보다 크다는 것)을 보여줌



f(n) = O(g(n))

O(Big-Oh)-표기

- $f(n) = 2n^2 - 8n + 3$
  - f(n)의 O-표기는 $O(n^2)$
  - f(n)의 단순화된 표현은 n^2
  - 단순화된 함수 n^2에 임의의 상수 c를 곱한 cn^2이 n이 증가함에 따라 f(n)의 상한이 됨 단, c > 0.



  - c=5일 때, $f(n) = 2n^2-8n+3$과 $5n^2$과의 교차점(n_0=1/3)이 생기는데, 이 교차점 이후 모든 n에 대해, 즉 n이 무한대로 증가할 때, $f(n) = 2n^2-8n+3$은 $5n^2$보다 절대로 커질 수 없음
  - 따라서 $O(n^2)$이 $2n^2-8n+3$의 점근적 상한이 됨

O-표기법 찾는 간단한 방법

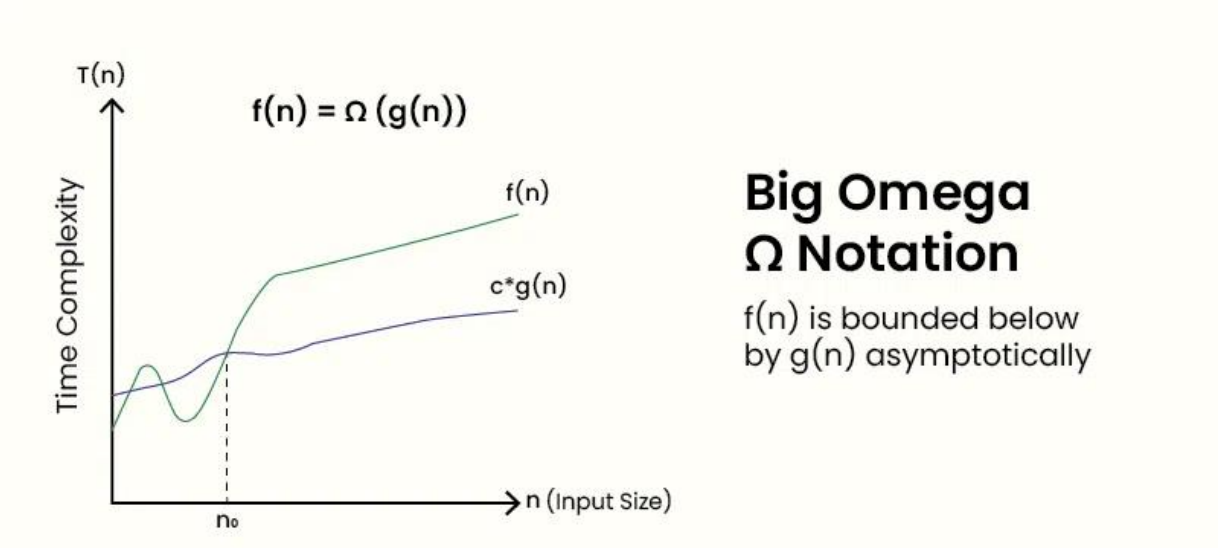
"다항식에서 최고 차수 항만을 취한 뒤, 그 항의 계수를 제거하여 g(n)을 정함"

(Big-Omega)-표기

점근적 하한 (Lower Bound)

- 알고리즘의 실행 시간이 "적어도 이만큼은 걸린다"는 것을 의미
- 알고리즘의 최선의 경우(Best Case) 성능의 한계를 명시할 때 사용

$$f(n) = \Omega(g(n))$$

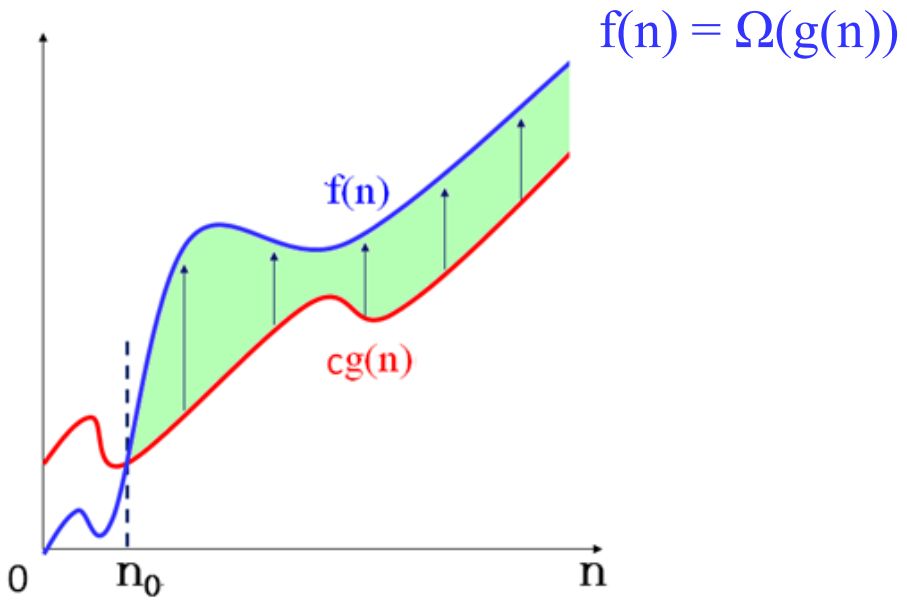


(Big-Omega)-표기

- Ω-표기의 정의
  - 모든 $n \geq n_0$에 대해서 $f(n) \geq cg(n)$이 성립하는 양의 상수 c와 n_0가 존재하면, f(n) = (g(n))
- Ω-표기의 의미
  - n_0 보다 큰 모든 n 대해서 f(n)이 cg(n)보다 작지 않다는 것
- f(n) = Ω(g(n))은 양의 상수를 곱한 g(n)이 f(n)에 미치지 못한다는 뜻
- g(n)을 f(n)의 하한(Lower Bound)이라 함

(Big-Omega)-표기

- $f(n) = \Omega(g(n))$
  - **n이 증가함에 따라 $\Omega(g(n))$이 점근적 하한이라는 것 (즉, g(n)이 n_0보다 큰 모든 n에 대해서 항상 f(n)보다 작다는 것)을 보여줌**

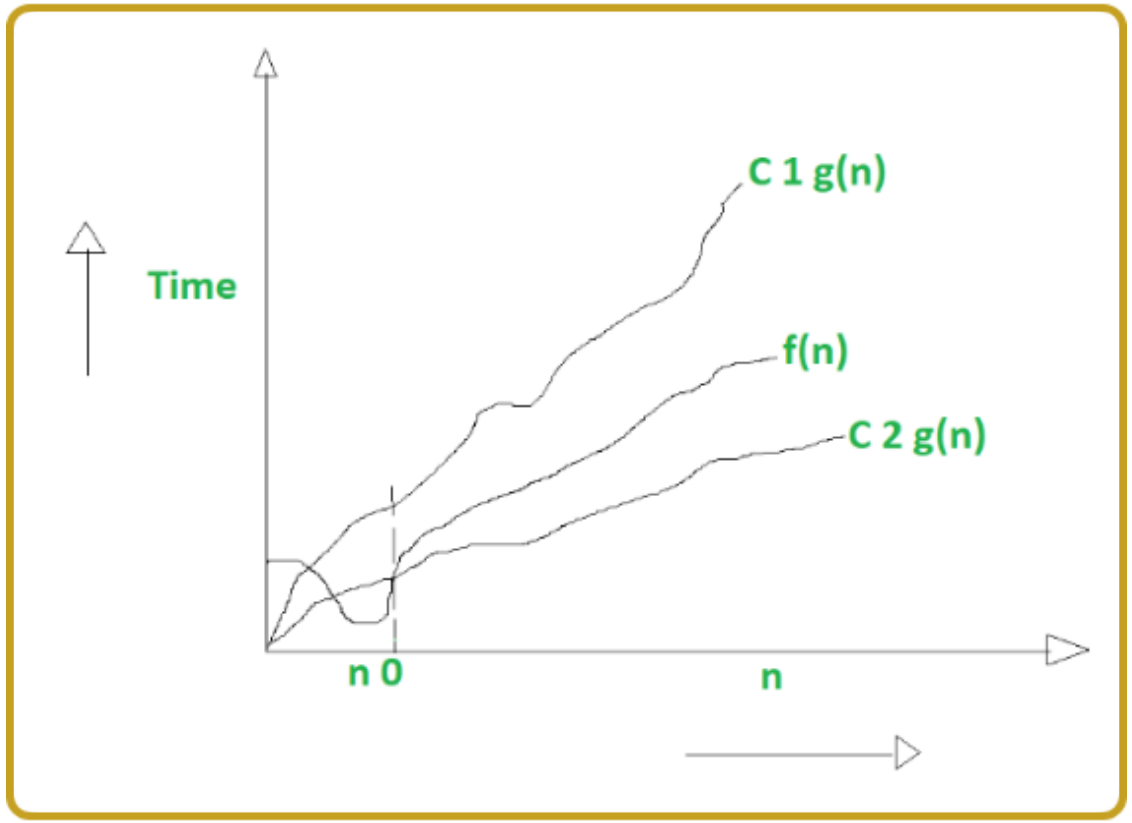


(Theta)-표기

평균적 성능의 지표

- 상한(O)과 하한( )이 동일할 때 성립하며, 알고리즘의 정확한 증가율을 나타냄
- "이 알고리즘은 정확히 이 정도로 수렴한다"는 가장 강력한 표기법

$$f(n) = \Theta(g(n))$$

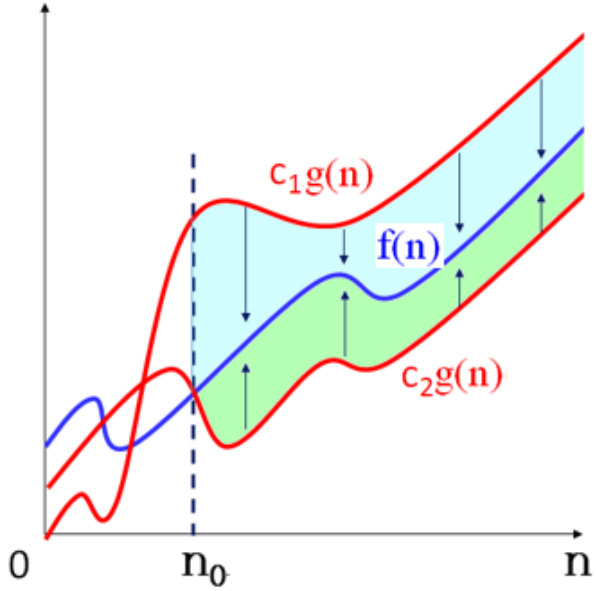


(Theta)-표기

- Θ-표기의 정의
  - 모든 $n \geq n_0$에 대해서 $c_1 g(n) \geq f(n) \geq c_2 g(n)$이 성립하는 양의 상수 c_1, c_2, n_0가 존재하면, $f(n) = \Theta(g(n))$이다.
- Θ-표기의 의미
  - 수행시간의 O-표기와 Ω-표기가 동일한 경우에 사용한다. 즉 동일한 증가율을 의미
- $2n^2+3n+5=O(n^2)$과 동시에 $2n^2+3n+5=\Omega(n^2)$ 이므로, $2n^2+3n+5=\Theta(n^2)$
- (n^2)은 n^2과 $(2n^2+3n+5)$이 유사한 증가율을 가지고 있다는 뜻
  - 따라서 $2n^2+3n+5 \neq \Theta(n^3)$ 이고, $2n^2+3n+5 \neq \Theta(n)$

(Theta)-표기

- $f(n)=\Theta(g(n))$
  - **n_0보다 큰 모든 n에 대해서 Θ-표기가 상한과 하한을 동시에 만족한다는 것을 보여줌**



$f(n) = \Theta(g(n))$

O(Big-Oh)-표기 예

- $n \geq 2,\ 3n+2 \leq 4n \Rightarrow 3n+2=O(n)$
- $n \geq 3,\ 3n+3 \leq 4n \Rightarrow 3n+3=O(n)$
- $n \geq 10,\ 100n+6 \leq 101n \Rightarrow 100n+6=O(n)$
- $n \geq 5,\ 10n^2+4n+2 \leq 11n^2 \Rightarrow 10n^2+4n+2=O(n^2)$
- $n \geq 4,\ 6*2^n+n^2 \leq 7*2^n \Rightarrow 6*2^n+n^2=O(2^n)$
- $n \geq 2,\ 3n+3 \leq 3n^2 \Rightarrow 3n+3=O(n^2)$
- $n \geq 2,\ 10n^2+4n+2 \leq 10n^4 \Rightarrow 10n^2+4n+2=O(n^4)$

(Big-Omega)-표기 예

- $n \geq 1$, $3n+2 \geq 3n \Rightarrow 3n+2 = \Omega(n)$
- $n \geq 1$, $3n+3 \geq 3n \Rightarrow 3n+3 = \Omega(n)$
- $n \geq 1$, $100n+6 \geq 100n \Rightarrow 100n+6 = \Omega(n)$
- $n \geq 1$, $10n^2+4n+2 \geq n^2 \Rightarrow 100n^2+4n+2 = \Omega(n^2)$
- $n \geq 1$, $6*2^n+n^2 \geq 2^n \Rightarrow 6*2^n+n^2 = \Omega(2^n)$

(Theta)-표기 예

- $n \geq 2$, $3n \leq 3n+2 \leq 4n \Rightarrow 3n+2 = \Theta(n)$
  - $c_1=3$, $c_2=4$, $n_0=2$
- $3n+3 = \Theta(n)$
- $10n^2+4n+2 = \Theta(n^2)$
- $6*2^n+n^2 = \Theta(2^n)$

표기법 간의 상관관계 요약

표기법	의미 (Intuition)	관계식	비유
Big-O (O)	점근적 상한	$f(n) \le c \cdot g(n)$	"최대 ~만큼" (≤)
Big-Ω (Ω)	점근적 하한	$f(n) \ge c \cdot g(n)$	"최소 ~만큼" (≥)
Big-Θ (Θ)	긴밀한 경계	$c1 \cdot g(n) \le f(n) \le c2 \cdot g(n)$	"정확히 ~만큼" (=)

자주 사용하는 O-표기

- O(1) 상수 시간(Constant time)
  - 입력 크기에 관계없이 항상 일정한 시간 소요.
  - 예: 배열 인덱스 접근, 해시 맵 조회.
- O(logn) 로그 시간(Logarithmic time)
  - 매 단계에서 검색 범위를 절반씩 줄여나감.
  - 예: 이진 탐색 (Binary Search).
- O(n) 선형 시간(Linear time)
  - 데이터 양에 비례하여 시간이 증가.
  - 예: 단순 루프를 통한 리스트 순회

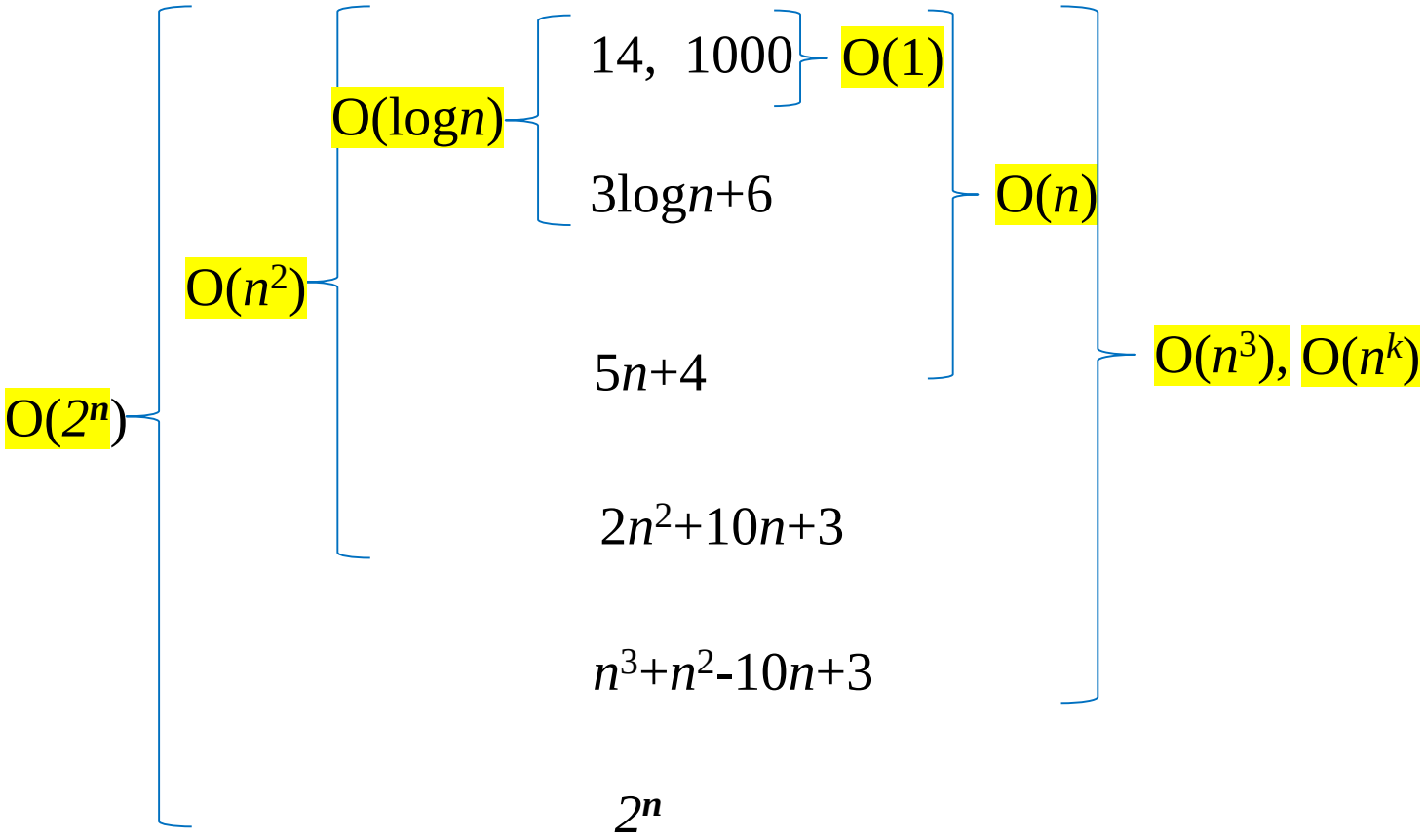
자주 사용하는 O-표기

- O(nlogn) 로그 선형 시간(Log-linear time)
  - 효율적인 정렬 알고리즘의 표준 속도.
  - 예: 병합 정렬 (Merge Sort), 퀵 정렬
- $O(n^2)$ 이차 시간(Quadratic time)
  - 데이터가 증가함에 따라 시간이 제곱으로 증가
  - 예: 이중 루프, 버블 정렬
- $O(2^n)$ 지수 시간 (Exponential time)
  - 입력이 늘어날 때마다 시간이 두 배씩 증가.
  - 예: 피보나치 재귀, 외판원 문제(TSP).

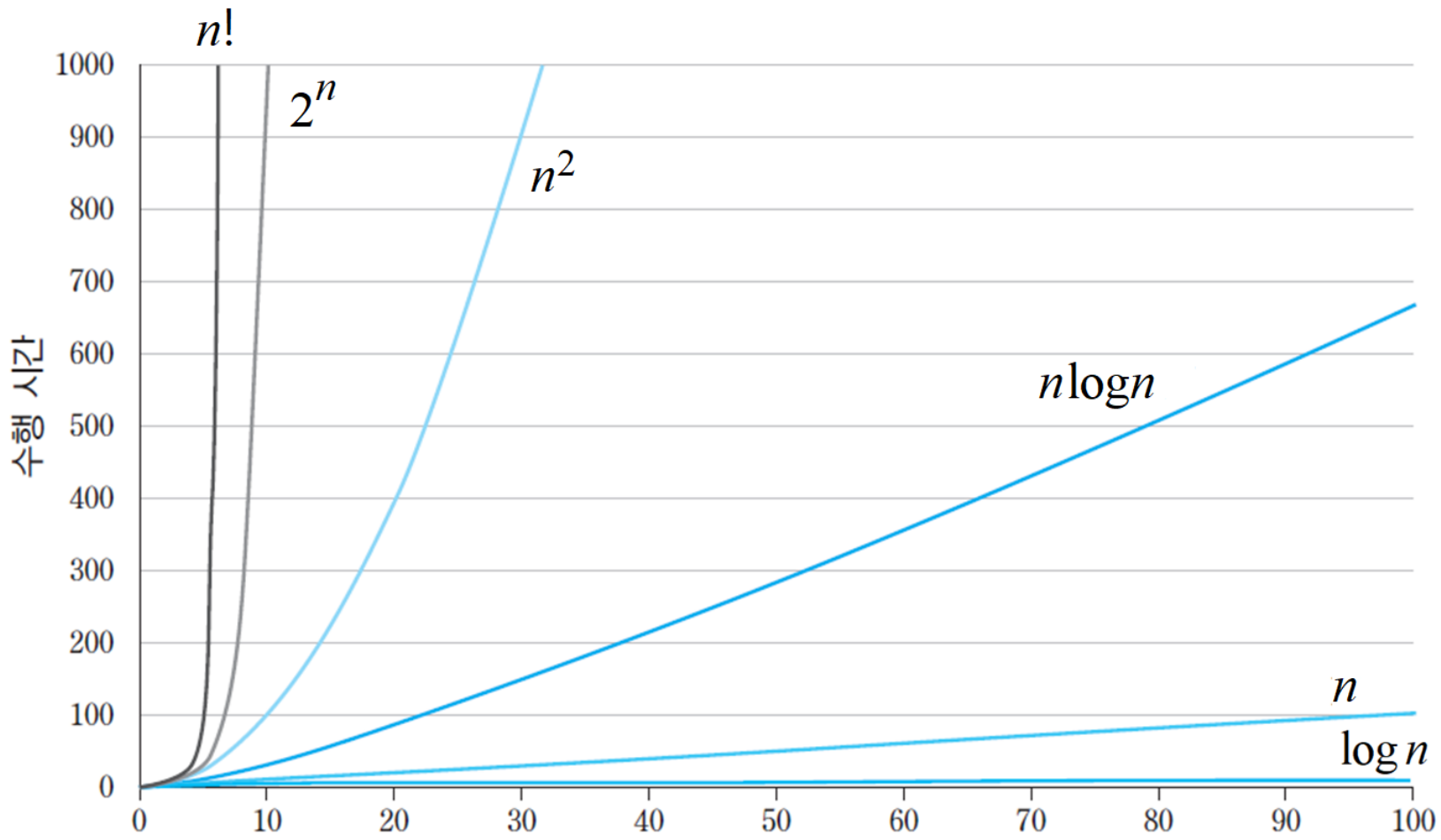
자주 사용하는 O-표기

- $O(n^3)$ 3차 시간(Cubic time)
- $O(n^k)$ 다항식 시간(Polynomial time), k는 상수

O-표기의 포함 관계


O(2^n)
O(n^2)
O(logn)
14, 1000
O(1)
3logn+6
O(n)
5n+4
O(n^3), O(n^k)
2n^2+10n+3
n^3+n^2-10n+3
2^n

함수의 증가율 비교



$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

경우에 따른 분석(Cases Analysis)

- **Worst Case (최악):** 가장 긴 실행 시간 보장

  (일반적인 기준)

- **Best Case (최선):** 가장 운이 좋은 경우 (현실성 낮음)

- **Average Case (평균):** 확률 분포를 고려한 기대 시간

  (계산 복잡)

- 소프트웨어 안정성을 위해 **최악의 경우 분석이 가장 중요하게 다뤄짐**

2.7 효율적 알고리즘의 필요성

- **10억개를 정렬하는데 PC에서** $O(n^2)$ **알고리즘은 300년,** $O(n\log n)$ **알고리즘은 5분**

$O(n^2)$	1,000	1백만	10억
PC	< 1초	2시간	300년
슈퍼컴	< 1초	1초	1주일

O(nlogn)	1,000	1백만	10억
PC	< 1초	<1초	5분
슈퍼컴	< 1초	<1초	<1초

효율적 알고리즘의 필요성

- 효율적인 알고리즘은 슈퍼 컴퓨터보다 더 큰 가치가 있음
- 값 비싼 H/W 기술 개발보다 효율적인 알고리즘 개발이 훨씬 더 경제적임

요약

- 알고리즘이란 문제를 해결하는 단계적 절차 또는 방법

- 알고리즘의 일반적인 특성
  - 정확성: 주어진 입력에 대해 올바른 해를 주어야
  - 수행성: 각 단계는 컴퓨터에서 수행 가능하여야.
  - 유한성: 유한 시간 내에 종료되어야
  - 효율성: 효율적일수록 그 가치가 높다.

요약

- 알고리즘은 대부분 의사 코드(pseudo code) 형태로 표현
- 알고리즘의 효율성은 주로 시간 복잡도 (Time Complexity)가 사용
- 시간 복잡도는 알고리즘이 수행하는 기본적인 연산 횟수를 입력 크기에 대한 함수로 표현
- 알고리즘의 복잡도 표현 방법:
  - 최악 경우 분석(Worst case Analysis)
  - 평균 경우 분석(Average case Analysis)
  - 최선 경우 분석(Best case Analysis)

요약

- **점근적 표기(Asymptotic Notation): 입력 크기 n이 무한대로 커질 때의 복잡도를 간단히 표현하기 위해 사용하는 표기법**

  1. 알고리즘의 본질적인 효율성 계층을 이해하는 강력한 도구임
  2. 상세한 구현 전에 성능 병목 지점을 예측할 수 있게 해줌
  3. 상용 소프트웨어 개발 시 스케일러빌리티(Scalability)를 보장하는 기초가 됨

- O-(Big-Oh) **표기: 점근적 상한**
- Ω-(Big-Omega) **표기: 점근적 하한**
- Θ-(Theta) **표기: 동일한 증가율**

Q & A